Software Engineering + AI Roadmap

A Complete, Week-by-Week Path From Beginner to Employable Software Engineer (2026 Edition)

Personalized Curriculum

July 2026

Contents

How To Use This Roadmap

Duration: 46 weeks (~10.5 months), assuming 12-15 hours/week dedicated to this roadmap, on top of the DSA/LeetCode practice you are already doing separately (which is intentionally excluded from this plan).

Stack philosophy: This roadmap uses the stack that startups and product companies are actually hiring for in 2026 — TypeScript-first, React + Vite, Node.js/Express, PostgreSQL + Prisma, modern auth, Docker, GitHub Actions, and AI-assisted engineering (Claude/Copilot/Cursor, LLM APIs, RAG). It intentionally excludes Linux/WSL (you're on Windows) and daily DSA (you already have that covered).

Structure: 8 phases, 46 weeks. Every week lists what you build, what you learn, why it matters, a detailed topic breakdown, learning order, mandatory vs. optional topics, common mistakes, free resources, and a time estimate. Every week also has a **mini project** and a **capstone update**.

The Capstone Project — "TaskForge AI": One evolving, production-grade project that grows from a static HTML page in Week 7 to a deployed, tested, AI-powered SaaS-style project management tool by Week 46. It is a task/project management application (think a mini Linear/Trello) with AI-assisted features (AI task breakdown, AI project summaries, AI-powered search over project docs using RAG). It touches frontend, backend, database, auth, APIs, testing, deployment, and AI — everything on your requirement list.

Golden rule: No week hands you source code for projects. You get requirements and specs only, because struggling productively is how the skill is actually built. If you get stuck for more than ~45–60 minutes, that's when you use AI tools (Week 39 teaches you how) — not before you've genuinely tried.

Phase Overview

Phase	Weeks	Focus
0	Week 1	Environment Setup & Dev Workflow
1	Weeks 1–6	JavaScript Mastery + TypeScript Foundations
2	Weeks 7–16	Frontend Engineering (HTML/CSS/React/TS)
3	Weeks 17–26	Backend Engineering (Node/Express/DBs/Auth)
4	Weeks 27–31	Software Engineering Craft (Clean Code, Patterns, Architecture, Networking)
5	Weeks 32–34	System Design, Security, Performance
6	Weeks 35–38	Cloud, Deployment, CI/CD, Docker
7	Weeks 39–43	AI Engineering for Software Engineers
8	Weeks 44–46	Capstone Finalization, Portfolio, Interview Prep

Phase 1 — JavaScript Mastery + TypeScript Foundations (Weeks 1–6)

Why this phase exists: Everything else — React, Node, testing, even AI SDKs — is JavaScript/TypeScript underneath. Weak JS fundamentals is the #1 reason self-taught developers stall in interviews and in real codebases. This phase over-invests here on purpose.

Week 1 — Windows Dev Environment + JavaScript Core

Build: A "JS Playground" repo with 10 small scripts (temperature converter, tip calculator, palindrome checker, etc.) run from the terminal with Node.

Capstone update: Create the `taskforge-ai` GitHub repo. Add a `README.md` with a one-paragraph project vision, an MIT license, and a `.gitignore`.

Learn: Core JavaScript syntax and the Windows-native tooling you'll use all roadmap long.

Why: You can't build anything reliably without a stable local setup and the actual language fundamentals — not "I've seen it in tutorials" familiarity, but "I can write it from a blank file" familiarity.

Detailed topics: 1. **Environment setup (Windows-native, no WSL needed)** - Node.js LTS via `nvm-windows` (why version managers matter) - VS Code setup: extensions (ESLint, Prettier, GitLens, Error Lens), settings sync - Windows Terminal + PowerShell basics (cd, ls/dir equivalents, running scripts) - npm basics: `npm init`, `package.json` anatomy, semantic versioning (^, ~) 2. **Variables & data types** - `var` vs `let` vs `const` and why `var` is avoided - Primitive types: string, number, boolean, null, undefined, symbol, bigint - Type coercion and == vs === - Template literals 3. **Operators & control flow** - Arithmetic, comparison, logical operators - Truthy/falsy values, short-circuit evaluation (&&, `||`, `??`) - `if/else`, `switch`, ternary operator - Loops: `for`, `while`, `for...of`, `for...in` 4. **Functions** - Function declarations vs. expressions vs. arrow functions - Parameters, default parameters, rest parameters - Return values, implicit vs explicit return - Understanding `this` in different contexts (regular function vs arrow function) 5. **Scope** - Global, function, and block scope - Hoisting (what actually gets hoisted, and what doesn't) - The Temporal Dead Zone

Recommended learning order: Environment setup → variables/types → operators/control flow → functions → scope.

Mandatory: All of the above. **Optional:** Symbol/bigint deep dive (nice-to-know, rarely used day-to-day).

Common beginner mistakes: - Using `var` out of habit, causing scope bugs later. - Comparing values with == instead of === and getting surprised by coercion. - Not understanding that arrow functions don't have their own `this` (this bites hard once you hit React). - Skipping terminal fluency and doing everything by clicking — this slows every future week down.

Free resources: MDN Web Docs (JavaScript Guide), "JavaScript.info" (free, thorough), freeCodeCamp's JavaScript curriculum, Node.js official docs.

Estimated time: 10–12 hours.

Week 2 — JavaScript Intermediate: Arrays, Objects, Closures

Build: A command-line "Contact Book" (add/search/delete contacts stored in an array of objects, saved to a local JSON file with Node's `fs` module).

Capstone update: Write a plain-language spec document (`docs/spec.md`) for TaskForge AI: core entities (User, Project, Task), MVP feature list, and a rough data model sketch (no code yet — just planning like an engineer).

Learn: How to actually manipulate real-world data shapes and understand closures, which underpin hooks, callbacks, and most "clever" JS you'll read in real codebases.

Why: Almost all app data is arrays of objects. If array/object methods aren't second nature, every feature takes 3x longer.

Detailed topics: 1. **Arrays** - `map`, `filter`, `reduce`, `find`, `some`, `every`, `sort`, `forEach` - Mutating vs. non-mutating methods (why this matters for React later) - Destructuring arrays, spread operator on arrays 2. **Objects** - Object literals, nested objects, computed property names - `Object.keys`, `Object.values`, `Object.entries` - Destructuring objects (including renaming and defaults) - Spread/rest on objects, shallow vs. deep copy 3. **Closures** - What a closure actually is (function + its lexical environment) - Practical use cases: counters, memoization, private state - Common closure pitfalls (loop variable capture with `var` vs `let`) 4. **Higher-order functions** - Functions that take/return functions - Building your own simple `map`/`filter` to prove you understand them 5. **Node.js basics for this project** - `fs.readFileSync` / `writeFileSync`, `JSON.parse`/`stringify` - `process.argv` for simple CLI input

Recommended learning order: Arrays → objects → closures → higher-order functions → apply with `fs`.

Mandatory: All array/object methods, closures, destructuring. **Optional:** Writing your own polyfills for `map`/`filter` (great for interview depth, not required for shipping).

Common beginner mistakes: - Mutating arrays/objects directly instead of creating new ones (this will cause real bugs in React). - Confusing `map` (transforms, always returns same length) with `forEach` (side effects, returns nothing). - Not understanding why closures "remember" variables — leads to confusing bugs in loops/timers.

Free resources: JavaScript.info (Objects, Closures chapters), MDN Array/Object method references, "Just JavaScript" by Dan Abramov (free articles) for mental models.

Estimated time: 10–12 hours.

Week 3 — Asynchronous JavaScript

Build: A small script that fetches data from a free public API (e.g., a weather or Pokémon API) and prints a formatted report — first with callbacks, then rewritten with Promises, then with `async/await`, so you feel the evolution firsthand.

Capstone update: Refine TaskForge AI's `docs/spec.md` with a "how the frontend will talk to the backend" section — sketch out 4–5 REST endpoints you'll eventually build (`GET /projects`, `POST /tasks`, etc.), even though nothing exists yet.

Learn: How JavaScript actually handles time — the event loop, callbacks, Promises, and async/await — because every real app talks to a server or a database.

Why: Async is where most junior developers get stuck: unhandled promise rejections, race conditions, "why did my data not load in time." Understanding the event loop removes the mystery.

Detailed topics: 1. **The event loop (conceptual, not implementation trivia)** - Call stack, Web APIs/Node APIs, callback queue, microtask queue - Why `setTimeout(fn, 0)` doesn't run immediately 2. **Callbacks** - Callback-based async patterns (Node's `fs.readFile`, etc.) - Callback hell and why it's a real problem 3. **Promises** - Creating a Promise, `resolve`/`reject` -

`.then`, `.catch`, `.finally` - `Promise.all`, `Promise.race`, `Promise.allSettled` 4. **async/await** - Syntax and how it maps to Promises under the hood - Error handling with `try/catch` - Sequential vs. parallel async calls (a very common interview + real-bug topic) 5. **Fetching data** - The Fetch API, `fetch()` basics, response parsing (`.json()`) - HTTP status codes at a glance (200, 400, 401, 404, 500) - Error handling for network failures

Recommended learning order: Event loop concept → callbacks → Promises → async/await → fetch in practice.

Mandatory: Promises, async/await, fetch, basic error handling. **Optional:** Deep event-loop internals (libuv-level detail) — useful curiosity, not required for employability.

Common beginner mistakes: - Forgetting to `await` a promise and debugging a "[object Promise]" for an hour. - Not wrapping `await` calls in `try/catch`, causing unhandled crashes. - Accidentally running independent async calls sequentially with multiple `await`s instead of `Promise.all`.

Free resources: JavaScript.info (Promises, async/await chapters), "What the heck is the event loop anyway?" (Philip Roberts, free YouTube talk), MDN Fetch API guide.

Estimated time: 10–12 hours.

Week 4 — Modern JavaScript (ES6+) and Modules

Build: Refactor Weeks 1–3's scripts into ES modules (`import`/`export`), split into multiple files, and package them as a small reusable npm-style utility library.

Capstone update: Initialize the actual TaskForge AI frontend scaffold using Vite (plain JS for now — TypeScript comes in Week 5). Push the first real commit.

Learn: The modern JS syntax and module system used in every production codebase.

Why: Real codebases are never one file. You need to know how modules, imports, and modern syntax sugar actually work before touching React/Node tooling, which assumes this fluency.

Detailed topics: 1. **ES Modules** - `import`/`export` (named vs. default exports) - Module scope, why modules solve the "everything is global" problem - CommonJS (`require`/`module.exports`) vs. ES Modules — knowing both, since Node code you'll read uses both 2. **Destructuring & spread (deeper patterns)** - Nested destructuring, default values in destructuring - Using spread for immutable updates (a pattern you'll use constantly in React) 3. **Template literals & tagged templates (awareness level)** 4. **Optional chaining & nullish coalescing** - `?.` and `??` and where they prevent real bugs 5. **Iterators & generators (conceptual understanding)** - What makes something iterable - Basic generator syntax (`function*`, `yield`) — enough to read library code, not to write complex generators 6. **Classes (since some libraries and OOP-style code still use them)** - Class syntax, constructors, methods, inheritance, `extends`/`super` - Getters/setters

Recommended learning order: ES Modules → destructuring/spread review → optional chaining/nullish coalescing → classes → iterators/generators (light).

Mandatory: ES Modules, optional chaining, nullish coalescing, classes (basic). **Optional:** Generators/iterators beyond a conceptual level.

Common beginner mistakes: - Mixing CommonJS `require` and ES `import` in the same file and getting confused by errors. - Forgetting `default` vs `named` export/import mismatches. - Overusing classes where a plain object/function would be simpler (common overengineering habit).

Free resources: JavaScript.info (Modules, Classes chapters), MDN (Optional chaining, Nullish coalescing), Vite's official "Getting Started" docs.

Estimated time: 8–10 hours.

Week 5 — TypeScript Fundamentals

Build: Convert the Week 4 utility library from JS to TypeScript, adding types to every function and catching at least 3 real bugs the type system reveals.

Capstone update: Add TypeScript to the TaskForge AI Vite frontend (`vite --template react-ts` style setup, or migrate). Define the initial shared types: `User`, `Project`, `Task` as TypeScript interfaces in a `types/` folder.

Learn: TypeScript's type system from the ground up — this is now a baseline expectation at most product companies, not a "nice to have."

Why: TypeScript catches entire categories of bugs before runtime, makes refactoring safe, and is expected in virtually every modern JS/React/Node job posting in 2026.

Detailed topics: 1. **Why TypeScript** - What problems it solves vs. plain JS - How the TS compiler works (compile-time checking, `tsc`, type erasure at runtime) 2. **Basic types** - `string`, `number`, `boolean`, `array`, `tuple`, `any`, `unknown`, `void`, `never` - Why `any` defeats the purpose and `unknown` is the safer escape hatch 3. **Type inference vs. explicit typing** - When TypeScript can infer types and when you should be explicit 4. **Interfaces & type aliases** - `interface` vs `type` — differences and when to use each - Optional properties (`?`), readonly properties - Extending interfaces 5. **Functions in TypeScript** - Typing parameters and return values - Optional/default parameters, function overloads (awareness level) 6. **Union & intersection types** - `|` and `&`, discriminated unions (a pattern you'll use constantly) 7. **Type narrowing** - `typeof`, `instanceof`, truthiness checks, discriminated union narrowing

Recommended learning order: Why TS → basic types → inference → interfaces/type aliases → functions → unions/intersections → narrowing.

Mandatory: Everything except function overloads. **Optional:** Function overloads (rare in day-to-day app code).

Common beginner mistakes: - Sprinkling `any` everywhere to "make errors go away," which defeats the point of TypeScript entirely. - Confusing `interface` and `type` and being unable to explain the difference in an interview. - Not narrowing union types properly before using a property that only exists on one branch.

Free resources: "TypeScript for JS Programmers" (official TS handbook), "Total TypeScript" free tips (Matt Pocock), TypeScript official Playground for experimenting.

Estimated time: 10–12 hours.

Week 6 — TypeScript Advanced + Tooling

Build: A typed CLI "Expense Tracker" using generics for a reusable `Repository<T>` pattern (in-memory store with add/find/update/delete), demonstrating you can design a small type-safe system, not just annotate variables.

Capstone update: Set up ESLint + Prettier for TaskForge AI with a shared config, add a `tsconfig.json` with `strict: true`, and write a `CONTRIBUTING.md` documenting code style — treating the repo like a real product repo from day one.

Learn: Generics, utility types, and the tooling/config that makes TypeScript pleasant instead of painful.

Why: Generics and utility types are what separate "I can add `: string` to a variable" from "I can design reusable, type-safe systems" — the latter is what gets you hired.

Detailed topics: 1. **Generics** - Why generics exist (reusable, type-safe functions/classes) - Generic functions, generic interfaces, generic constraints (`extends`) - Practical examples: a generic `Repository<T>`, a generic `ApiResponse<T>` 2. **Utility types** - `Partial`, `Required`, `Pick`, `Omit`, `Record`, `Readonly` - When to reach for each in real code (e.g., `Partial<User>` for update payloads) 3. **Enums vs. union literal types** - Why many teams prefer string literal unions over TS `enum` in 2026 4. **`tsconfig.json` deep dive** - `strict` mode and what it actually enables - `target`, `module`, `esModuleInterop`, `paths`/aliases 5. **Tooling** - ESLint with the TypeScript plugin, Prettier, and how they divide responsibilities - Pre-commit hooks (Husky + lint-staged) — awareness level, set up once 6. **Type-only imports, ambient types, `.d.ts` files (awareness level)**

Recommended learning order: Generics → utility types → enums/unions → tsconfig → linting/formatting → type-only imports (light).

Mandatory: Generics, utility types, strict tsconfig, ESLint/Prettier setup. **Optional:** Writing custom `.d.ts` declaration files.

Common beginner mistakes: - Avoiding generics entirely and duplicating near-identical typed functions instead. - Turning off `strict` mode "to make errors go away" instead of fixing the underlying issue. - Not configuring ESLint/Prettier together, leading to conflicting rules and noisy diffs.

Free resources: TypeScript Handbook ("Generics," "Utility Types" sections), Prettier + ESLint official integration docs, Husky official docs.

Estimated time: 10 hours.

Phase 1 milestone check: You should now be able to write a small typed CLI tool from scratch, without a tutorial open, using arrays/objects/closures/async/generics comfortably. If not, spend a buffer week here before moving on — Phase 2 assumes this is solid.

Phase 2 — Frontend Engineering: HTML, CSS, React, TypeScript (Weeks 7–16)

Week 7 — HTML5 & Accessibility Fundamentals

Build: A fully static, semantic "TaskForge AI Landing Page" (hero, features, pricing placeholder, footer) using only HTML — no CSS framework yet.

Capstone update: This landing page becomes the first real UI artifact of TaskForge AI, committed to the repo under `apps/web` (or similar), even though it's not connected to React yet.

Learn: Semantic HTML and accessibility basics — skipped by most self-taught developers, checked in every serious code review.

Why: Recruiters and interviewers notice `<div>` soup instantly. Accessibility is a legal/business requirement at real companies, and semantic HTML is the foundation good CSS and React code is built on.

Detailed topics: 1. **Semantic HTML** - `header`, `nav`, `main`, `section`, `article`, `aside`, `footer` - Why semantic tags beat `<div>` for everything - Headings hierarchy (`h1`–`h6`) done correctly 2. **Forms** - `input` types, `label` + `for`/`id` association, `fieldset`/`legend` - Form validation attributes (`required`, `pattern`, `min`/`max`) 3. **Accessibility (a11y) basics** - Why alt text matters, ARIA roles (only where semantic HTML isn't enough) - Keyboard navigability, focus states - Color contrast basics 4. **Meta & document structure** - `<head>` essentials (viewport meta, charset, title, description) - Basic SEO awareness (not a deep SEO course — just the hygiene basics)

Recommended learning order: Document structure → semantic layout tags → forms → accessibility.

Mandatory: Semantic tags, forms, basic a11y (labels, alt text, focus). **Optional:** ARIA deep dive beyond basic roles.

Common beginner mistakes: - Wrapping everything in `<div>`s instead of semantic elements. - Forgetting `<label>` on form inputs, which breaks accessibility and UX. - Using `<div onClick>` instead of a real `<button>` (loses keyboard accessibility for free).

Free resources: MDN HTML guide, "Web.dev Learn HTML," "The A11Y Project" (free, practical).

Estimated time: 6–8 hours.

Week 8 — CSS Fundamentals: Box Model, Flexbox, Grid

Build: Style the Week 7 landing page fully responsive (mobile, tablet, desktop breakpoints) using plain CSS — no framework yet, so the fundamentals actually stick.

Capstone update: TaskForge AI landing page is now responsive and visually presentable — a genuine portfolio-ready artifact by itself.

Learn: The core CSS layout model every framework (including Tailwind) is built on top of.

Why: If you jump straight to Tailwind without understanding the box model/flexbox/grid underneath, you'll be stuck whenever a class doesn't do what you expect.

Detailed topics: 1. **The box model** - Content, padding, border, margin - `box-sizing: border-box` and why almost every project sets it globally 2. **Selectors & specificity** - Class, ID, element, combinator selectors - Specificity rules, the cascade, `!important` (and why to

avoid it) 3. **Flexbox** - `display: flex`, main/cross axis - `justify-content`, `align-items`, `flex-wrap`, `flex-grow/shrink/basis` - Common flex layout patterns (navbars, centered content, cards) 4. **CSS Grid** - `display: grid`, `grid-template-columns/rows`, `gap` - `grid-template-areas` for readable layouts - When to use Grid vs. Flexbox 5. **Responsive design** - Media queries, mobile-first vs. desktop-first - Relative units: `rem`, `em`, `%`, `vw/vh` vs. `px`

Recommended learning order: Box model → selectors/specificity → Flexbox → Grid → responsive/media queries.

Mandatory: All of the above. **Optional:** CSS Grid advanced named areas for complex dashboards (useful later, not urgent now).

Common beginner mistakes: - Fighting the box model instead of setting `border-box` globally. - Using Flexbox for everything, including 2D layouts that Grid handles far more cleanly. - Hardcoding pixel widths instead of responsive units, breaking on smaller screens.

Free resources: "CSS Flexbox Froggy" and "CSS Grid Garden" (free interactive games), MDN CSS Layout guide, Josh Comeau's free CSS articles.

Estimated time: 10 hours.

Week 9 — Modern CSS: Tailwind CSS + Component-Level Styling

Build: Re-style the landing page using Tailwind CSS from scratch, and build 3 reusable UI pieces (button variants, a card, a badge) as a tiny "design system" reference sheet.

Capstone update: Migrate TaskForge AI's landing page to Tailwind. Set up a `theme` config (colors, spacing scale) so the whole project has consistent design tokens from the start.

Learn: Utility-first CSS — the dominant styling approach at 2026 startups — plus enough CSS architecture thinking to not produce unmaintainable class soup.

Why: Tailwind is the most commonly used styling approach in modern React job postings. Knowing it (and *why* it works the way it does) is a practical hiring signal.

Detailed topics: 1. **Utility-first philosophy** - Why utility classes vs. traditional CSS/BEM, tradeoffs - Tailwind config: theme customization, spacing/color scales 2. **Core utilities** - Layout (flex/grid utilities), spacing, typography, color, borders, shadows - Responsive prefixes (`sm:`, `md:`, `lg:`) - State variants (`hover:`, `focus:`, `disabled:`) 3. **Component extraction patterns** - When to extract a React component vs. use `@apply` - Avoiding "class name spaghetti" in large components 4. **Basic animations & transitions in Tailwind** 5. **Dark mode basics (`dark:` variant) — optional but common in 2026 apps**

Recommended learning order: Philosophy/setup → core utilities → responsive/state variants → component extraction patterns → dark mode.

Mandatory: Setup, core utilities, responsive/state variants, component extraction thinking. **Optional:** Dark mode implementation, custom Tailwind plugins.

Common beginner mistakes: - Copy-pasting huge class strings into every element instead of extracting a component. - Fighting Tailwind's defaults instead of customizing the theme config once. - Never learning the underlying CSS, so debugging a broken layout becomes guesswork.

Free resources: Official Tailwind CSS docs (excellent and free), Tailwind's own "Refactoring UI" free blog posts.

Estimated time: 8 hours.

Week 10 — React Fundamentals

Build: A "Recipe Book" React app: list of recipes rendered from an array of objects, a details view, and a search filter — no routing or backend yet.

Capstone update: Scaffold TaskForge AI's real component structure: `Layout`, `ProjectCard`, `TaskList`, `TaskItem` as static components rendering hardcoded mock data.

Learn: React's component model and the mental model shift from "manipulating the DOM" to "describing UI as a function of state."

Why: React (or a React-like framework) remains the dominant frontend approach at product companies in 2026. This is the highest-leverage frontend skill you'll learn.

Detailed topics: 1. **JSX** - What it is, why it exists (syntax sugar over `React.createElement`) - Embedding expressions, conditional rendering (&&, ternary) - Rendering lists with `.map()` and the `key` prop (and why keys matter) 2. **Components** - Functional components (the only kind you'll write in 2026) - Component composition — building complex UIs from small pieces - Reusable, single-responsibility components 3. **Props** - Passing props, typing props (paired with TS in Week 14, plain JS understanding first) - `children` prop and composition patterns - Props best practices (avoid prop drilling too deep — foreshadowing Context) 4. **The React mental model** - Declarative vs. imperative UI - Re-rendering: what causes a component to re-render 5. **Styling in React** - Inline styles vs. CSS modules vs. Tailwind in components (pick one convention and stay consistent)

Recommended learning order: JSX → mental model → components → props → lists/keys → composition.

Mandatory: All of the above. **Optional:** `React.createElement` deep internals (good to know once, not to memorize).

Common beginner mistakes: - Using array index as `key` on lists that can reorder (causes subtle bugs). - Trying to directly mutate the DOM instead of trusting React's rendering. - Overly large "god components" instead of composing smaller ones.

Free resources: Official React docs (react.dev — rewritten and excellent), "React for Beginners" free sections on freeCodeCamp.

Estimated time: 10–12 hours.

Week 11 — React State & Core Hooks

Build: Turn the Recipe Book into an interactive app: favorite/unfavorite recipes, a live search box, and a "recently viewed" list — all driven by state.

Capstone update: Add real interactivity to TaskForge AI: creating a task (in local state for now), marking it complete, deleting it. No backend yet — this proves your component/state design before wiring up APIs.

Learn: `useState` and `useEffect` — the two hooks you'll use most, and the mental model of "state drives UI."

Why: State management is the core skill of frontend engineering. Getting `useEffect` wrong is the single most common source of React bugs in real codebases and interviews.

Detailed topics: 1. **State management with `useState`** - Initializing state, updating state (and why it's asynchronous/batched) - Updating state based on previous state (functional updates) - State with objects/arrays — the immutability requirement (ties back to Week 2) 2. **State patterns** - Lifting state up - Derived state vs. stored state (don't store what you

can compute) - Controlled components (forms driven by state) 3. **`useEffect`** - What side effects are and why they need a separate hook - Dependency arrays — the #1 source of React bugs, covered thoroughly - Cleanup functions (event listeners, subscriptions, timers) - Common `useEffect` anti-patterns (using it for things that don't need it) 4. **Component lifecycle understood through hooks (mount/update/unmount)**

Recommended learning order: `useState` basics → state update patterns → lifting state up → `useEffect` → dependency arrays deep dive → cleanup.

Mandatory: Everything. **Optional:** N/A — this week is foundational; nothing to skip.

Common beginner mistakes: - Missing dependencies in `useEffect`'s dependency array (or worse, silencing the ESLint warning). - Directly mutating state (`state.push(...)` instead of creating a new array). - Using `useEffect` to compute a derived value that could just be a plain variable.

Free resources: react.dev "State: A Component's Memory" and "Synchronizing with Effects" guides (genuinely excellent, written by the React team), "A Complete Guide to useEffect" by Dan Abramov (free article).

Estimated time: 12 hours.

Week 12 — Advanced Hooks & Context API

Build: Add a global "theme" (light/dark) and a global "toast notification" system to the Recipe Book using Context — deliberately choosing Context over prop-drilling to feel the difference.

Capstone update: Introduce a `TaskContext`/`ProjectContext` in TaskForge AI to share the active project and task list across components without prop drilling. Optimize the task list re-renders with `useMemo`.

Learn: `useMemo`, `useCallback`, `useReducer`, and the Context API — the hooks that solve state-sharing and performance problems at scale.

Why: Every non-trivial React app needs shared state and performance tuning. This is also a very common interview topic ("when would you use useMemo vs useCallback?").

Detailed topics: 1. **`useMemo` and `useCallback`** - What memoization solves (expensive recalculation, unnecessary child re-renders) - `useMemo` for derived values, `useCallback` for stable function references - When NOT to use them (premature optimization is a real anti-pattern) 2. **`useReducer`** - When reducer-based state beats multiple `useState` calls - Actions, reducer functions, dispatch — building a mental model that transfers to Redux later 3. **Context API** - Creating a context, Provider, `useContext` - When Context is the right tool vs. when it's overused - Performance pitfall: Context re-renders all consumers on any change 4. **Custom hooks** - Extracting reusable logic into custom hooks (`useLocalStorage`, `useDebounce`, etc.) - Rules of hooks (why they must be called unconditionally, at the top level)

Recommended learning order: `useMemo`/`useCallback` → `useReducer` → Context API → custom hooks (tying it together).

Mandatory: All of the above. **Optional:** Advanced Context performance patterns (splitting contexts) — good to know, revisit if it becomes a real bottleneck.

Common beginner mistakes: - Wrapping every function in `useCallback` "just in case," adding complexity with no benefit. - Putting everything into one giant Context, causing unnecessary re-renders app-wide. - Breaking the rules of hooks by calling them inside conditionals or loops.

Free resources: react.dev hooks reference, Kent C. Dodds' free articles on "When to use-Memo and useCallback," react.dev "Extracting State Logic into a Reducer."

Estimated time: 10–12 hours.

Week 13 — Routing & Forms

Build: Turn the Recipe Book into a multi-page app with React Router (list page, detail page, add-recipe page with a real validated form).

Capstone update: Add React Router to TaskForge AI: `/login`, `/dashboard`, `/projects/:id`, `/settings` routes, with a protected-route wrapper pattern (even before real auth exists — build the pattern now).

Learn: Client-side routing and robust form handling — two things every real app needs and most tutorials handle poorly.

Why: Any app with more than one screen needs routing. Forms are where most user input (and most bugs/UX complaints) happen.

Detailed topics: 1. **React Router fundamentals** - `BrowserRouter`, `Routes`, `Route`, nested routes - `Link`/`NavLink` vs. `<a>` tags (why client-side navigation matters) - URL params (`useParams`), query params (`useSearchParams`), programmatic navigation (`useNavigate`) 2. **Protected routes** - Building a route guard pattern for authenticated-only pages - Redirect patterns, loading states while auth status is unknown 3. **Forms in React** - Controlled vs. uncontrolled inputs - Handling multiple form fields cleanly - Client-side validation strategies 4. **Form libraries (introductory)** - Why libraries like React Hook Form exist (performance + less boilerplate) - Basic React Hook Form usage: `register`, `handleSubmit`, error display - Schema validation with Zod paired with React Hook Form (foreshadows backend validation in Week 23)

Recommended learning order: Router basics → nested/dynamic routes → protected routes pattern → controlled forms → React Hook Form + Zod.

Mandatory: Router fundamentals, protected routes, controlled forms. **Optional:** Uncontrolled inputs deep dive (React Hook Form abstracts most of this away).

Common beginner mistakes: - Using `<a href>` for internal navigation, causing full page reloads. - Re-implementing form state manually for every form instead of learning React Hook Form once. - Forgetting to handle the "loading" state for protected routes, causing a flash of the wrong screen.

Free resources: Official React Router docs, React Hook Form official docs + free tutorial, Zod official docs.

Estimated time: 10 hours.

Week 14 — React + TypeScript + API Integration (React Query)

Build: Convert the Recipe Book to TypeScript, and connect it to a free public API (e.g., a recipe or food API) using TanStack Query instead of raw `useEffect` + `fetch`.

Capstone update: Convert all TaskForge AI components to TypeScript with proper prop types. Since the real backend doesn't exist yet, set up a mock API (e.g., a local JSON server or MSW — Mock Service Worker) and wire TanStack Query to it, so the data-fetching layer is real and ready for Week 17+.

Learn: Typed React components, and TanStack Query — the modern standard for server-state management (replacing manual `useEffect` data-fetching in most 2026 codebases).

Why: Typing props/state prevents a huge class of runtime bugs. Raw `useEffect` data-fetching has well-known problems (race conditions, no caching, no retries) that React Query solves — and it's now expected knowledge in React job postings.

Detailed topics: 1. **TypeScript with React** - Typing props (`interface Props`), typing `useState`, typing event handlers - Typing children, typing custom hooks' return values - Generic components (light touch — e.g., a generic `<List<T>>` component) 2. **API integration fundamentals** - Loading/error/success UI states (the three states every data-fetching UI needs) - Why manual `useEffect` fetching gets messy at scale (race conditions, no caching) 3. **TanStack Query (React Query)** - `useQuery` for fetching, `useMutation` for writes - Query keys, caching, automatic refetching, stale time - Optimistic updates (conceptual understanding) - DevTools for inspecting cache state 4. **Mocking APIs during development** - Why you mock a backend before it exists (parallelizes frontend/backend work) - Basics of MSW or a local JSON server

Recommended learning order: TS with React → loading/error/success states → React Query basics (`useQuery`) → mutations (`useMutation`) → mocking setup.

Mandatory: TS typing for props/state, `useQuery`/`useMutation` basics, loading/error UI states. **Optional:** Optimistic updates (powerful, but fine to add later once basics are solid).

Common beginner mistakes: - Managing server data in `useState` + `useEffect` when React Query would be simpler and more correct. - Forgetting to handle the error state, leaving users staring at a blank/broken screen. - Using `any` to "get TypeScript to stop complaining" on API response types instead of defining a proper interface.

Free resources: Official TanStack Query docs (excellent, has a free course track), React TypeScript Cheatsheet (community-maintained, free), MSW official docs.

Estimated time: 12 hours.

Week 15 — State Management at Scale + Performance Optimization

Build: A "Kanban Board" mini project (drag-free version: buttons to move cards between To Do/In Progress/Done columns) using Zustand for global state.

Capstone update: Introduce Zustand (or Redux Toolkit — pick one and justify the choice in `docs/spec.md`) for TaskForge AI's global UI state (active project, filters, modal state). Profile the app with React DevTools Profiler and fix at least one real unnecessary re-render.

Learn: When Context isn't enough, dedicated state management libraries, and how to actually measure and fix React performance issues (instead of guessing).

Why: Interviewers ask "how would you manage state in a large app" constantly. Being able to answer with real experience — not just Context — is a strong signal.

Detailed topics: 1. **When you need more than Context** - Symptoms: too many providers, unrelated re-renders, complex update logic 2. **Zustand fundamentals** - Creating a store, selectors, actions - Why Zustand avoids Context's re-render problem 3. **Redux Toolkit (awareness + basics, since many companies still use it)** - Slices, reducers, actions, `configureStore` - `useSelector`/`useDispatch` - RTK Query as an alternative to React Query (awareness level) 4. **Performance optimization** - React DevTools Profiler — actually using it to find real bottlenecks - `React.memo` and when it helps vs. does nothing - Code splitting with `React.lazy` + `Suspense` - Virtualization for long lists (conceptual — e.g., `react-window`, awareness level)

Recommended learning order: When to reach for global state → Zustand → Redux Toolkit basics (awareness) → profiling → memoization → code splitting.

Mandatory: One state management library in depth (Zustand recommended for speed of learning), profiling, `React.memo` basics, code splitting basics. **Optional:** Redux Toolkit mastery (fine to be "aware of it" rather than expert, unless a job specifically requires it), list virtualization.

Common beginner mistakes: - Reaching for Redux/Zustand before Context/local state is even insufficient (premature complexity). - Wrapping every component in `React.memo` without profiling first — sometimes it makes things worse. - Never opening the Profiler and just guessing at performance problems.

Free resources: Official Zustand docs, official Redux Toolkit "Essentials" tutorial (free), react.dev "Render and Commit" + performance docs.

Estimated time: 12 hours.

Week 16 — Frontend Testing + Capstone Frontend Polish

Build: Write a full test suite (unit + component tests) for the Kanban Board mini project from Week 15 using Vitest + React Testing Library.

Capstone update: Write tests for TaskForge AI's core components (`TaskItem`, `ProjectCard`, form validation logic). Do a full UI polish pass: consistent spacing, empty states, loading skeletons, error boundaries. This closes out the frontend phase with a genuinely presentable, tested UI (still running against mock data).

Learn: How to test React components the way real teams do — behavior-focused, not implementation-detail-focused.

Why: "No tests" is one of the fastest ways a portfolio project gets dismissed by an experienced interviewer. Testing is also a concrete, provable skill you can speak to in interviews.

Detailed topics: 1. **Testing philosophy** - Unit vs. integration vs. end-to-end (E2E) tests, and where component tests fit - Testing behavior ("what the user sees/does") vs. implementation details 2. **Vitest fundamentals** - `describe`, `it`/`test`, `expect`, matchers - Setting up Vitest in a Vite project 3. **React Testing Library** - `render`, `screen`, queries (`getByRole`, `getByText`, `findBy...`) - Simulating user interaction with `userEvent` - Testing async UI (loading → data states) 4. **Mocking** - Mocking API calls (with MSW, tying back to Week 14) - Mocking modules/functions with Vitest's mocking utilities 5. **Error boundaries (React concept, tested here)** - What they catch, how to implement a basic one 6. **UI polish patterns** - Loading skeletons, empty states, and error states as first-class UI, not afterthoughts

Recommended learning order: Testing philosophy → Vitest basics → RTL queries/interactions → async testing → mocking → error boundaries → polish pass.

Mandatory: Vitest + RTL basics, testing at least core components, one error boundary. **Optional:** Full E2E testing with Playwright/Cypress (introduced conceptually now, hands-on optional — can revisit post-roadmap).

Common beginner mistakes: - Testing implementation details (internal state) instead of what the user actually sees/does — brittle tests that break on harmless refactors. - Not testing error/loading states at all, only the "happy path." - Skipping tests entirely because "the deadline is close" — this is exactly the habit to break now.

Free resources: Official Vitest docs, official React Testing Library docs + "Common Mistakes" guide by Kent C. Dodds (free, highly recommended), Testing Library's own philosophy

doc.

Estimated time: 10–12 hours.

Phase 2 milestone check: By the end of Week 16 you have a fully responsive, typed, tested, reasonably polished frontend for TaskForge AI running against mock data — genuinely something you could screen-record and put in a portfolio today, even before the backend exists.

Phase 3 — Backend Engineering (Weeks 17–26)

Week 17 — Node.js Fundamentals

Build: A CLI tool that reads a folder of text files and generates a word-frequency report, exercising Node's module system, file system, and streams (basic level).

Capstone update: Initialize the real TaskForge AI backend (`apps/api`) as a TypeScript Node project: `package.json`, `tsconfig.json`, folder structure (`src/routes`, `src/controllers`, `src/services` — even if empty, establishing the architecture now).

Learn: How Node.js works as a runtime — separate from "how to use Express" — because understanding the runtime is what lets you debug real production issues later.

Why: Node is the dominant backend runtime for JS/TS teams. Treating it as "just Express" leaves gaps that show up the moment something goes wrong in production (memory leaks, blocking the event loop, etc.).

Detailed topics: 1. **Node.js runtime model** - Single-threaded event loop (revisited from Week 3, now in a server context) - libuv, the thread pool, non-blocking I/O - CommonJS vs. ESM in Node specifically, and how to configure Node/TS for ESM 2. **Core modules** - `fs` (sync vs. async vs. promise-based APIs) - `path`, `os`, `process` (env vars, `process.argv`, exit codes) - `events` — the `EventEmitter` pattern (foundational for understanding much of Node's ecosystem) 3. **npm ecosystem** - `dependencies` vs. `devDependencies`, lockfiles and why they matter - Semantic versioning in practice, `npm scripts` 4. **Streams (conceptual + basic hands-on)** - Why streams exist (processing large data without loading it all into memory) - Readable/Writable streams at a basic level 5. **Environment variables & config** - `.env` files, `dotenv`, never committing secrets (tie to Git hygiene)

Recommended learning order: Runtime model → core modules (`fs`/`path`/`process`) → npm ecosystem → EventEmitter → streams (light) → env config.

Mandatory: Runtime model, core modules, npm fundamentals, env config. **Optional:** Deep stream API mastery (revisit if/when you build file-upload/processing features).

Common beginner mistakes: - Using blocking sync `fs` calls in what will become a server (fine for scripts, wrong for servers). - Committing `.env` files with real secrets to Git. - Not understanding why a long synchronous loop can freeze an entire Node server (single-threaded event loop).

Free resources: Official Node.js docs, "Node.js Design Patterns" free chapter previews, freeCodeCamp Node.js curriculum.

Estimated time: 10 hours.

Week 18 — Express.js & Building REST APIs

Build: A standalone "Bookstore API" (CRUD for books, in-memory data first) using Express + TypeScript, following REST conventions properly.

Capstone update: Build TaskForge AI's first real endpoints: `GET/POST /projects`, `GET/POST/PATCH/DELETE /tasks` — in-memory for now (a real database comes in Weeks 19–20). Connect the frontend's mock API layer to these real endpoints.

Learn: Express fundamentals and REST API design conventions used across the industry.

Why: This is the moment TaskForge AI becomes a real full-stack app. REST conventions are assumed knowledge in virtually every backend interview.

Detailed topics: 1. **Express fundamentals** - App setup, routing (`app.get/post/put/patch/delete`) - Route parameters, query parameters, request body parsing (`express.json()`) - Middleware — what it is, the request/response/next cycle 2. **REST API design** - Resource-based URL design (`/projects/:id/tasks`, not `/getTasksForProject`) - HTTP methods mapped correctly to CRUD operations - HTTP status codes used correctly (201 vs 200, 204, 400 vs 422, 401 vs 403, 404, 500) - Consistent response shapes (success and error envelopes) 3. **Middleware patterns** - Built-in vs. custom middleware, third-party middleware (`cors`, `morgan`/logging, `helmet` for security headers) - Error-handling middleware (the 4-argument signature) 4. **Project structure** - Separating routes, controllers, and (stubbed) services from day one - Why this separation matters as the app grows

Recommended learning order: Express basics → routing/params → middleware concept → REST conventions/status codes → error-handling middleware → project structure.

Mandatory: All of the above. **Optional:** Advanced middleware composition patterns (fine to learn as needed).

Common beginner mistakes: - Returning 200 for everything, including errors — breaks client error handling. - Putting all logic directly in route handlers instead of separating concerns (becomes unmaintainable fast). - Forgetting `express.json()` and being confused why `req.body` is undefined.

Free resources: Official Express.js docs, "REST API Tutorial" (restfulapi.net, free), MDN HTTP status code reference.

Estimated time: 12 hours.

Week 19 — Databases I: SQL & PostgreSQL Fundamentals

Build: Design and write raw SQL (no ORM yet) for a small library system: tables for `books`, `authors`, `borrowers`, with proper foreign keys, and 10 practice queries (joins, aggregates, filtering).

Capstone update: Design TaskForge AI's actual relational schema on paper/diagram first: `users`, `projects`, `tasks`, `project_members` — including primary keys, foreign keys, and relationship types (one-to-many, many-to-many). Set up a local PostgreSQL instance (via Docker Desktop, previewing Week 38, or a native Windows installer).

Learn: Relational database fundamentals and SQL — the skill that underlies every ORM you'll ever use, and a very common interview topic.

Why: ORMs hide SQL until they don't — the moment you need a complex query or hit a performance problem, you need real SQL knowledge. Interviewers frequently test raw SQL directly.

Detailed topics: 1. **Relational database concepts** - Tables, rows, columns, primary keys, foreign keys - Relationship types: one-to-one, one-to-many, many-to-many (with join tables) - Normalization basics (1NF/2NF/3NF — conceptual understanding, not academic memorization) 2. **SQL fundamentals** - `SELECT`, `WHERE`, `ORDER BY`, `LIMIT` - `INSERT`, `UPDATE`, `DELETE` - `JOIN` types: `INNER`, `LEFT`, (awareness of `RIGHT`/`FULL`) - Aggregate functions: `COUNT`, `SUM`, `AVG`, `GROUP BY`, `HAVING` 3. **Constraints & indexes** - `NOT NULL`, `UNIQUE`, `DEFAULT`, `CHECK` - What an index is and why it speeds up queries (conceptual, not internals-heavy) 4. **Transactions (conceptual)** - `BEGIN`/`COMMIT`/`ROLLBACK`, why atomicity matters (e.g., a money transfer or multi-table update) 5. **PostgreSQL setup** - Installing/running Postgres locally, using a GUI client (e.g., TablePlus, pgAdmin) to inspect data

Recommended learning order: Relational concepts → SQL DML (`SELECT/INSERT/UPDATE/DELETE`) → joins → aggregates → constraints/indexes → transactions (concept) → local setup.

Mandatory: Everything except deep normalization theory. **Optional:** Formal normalization theory (1NF–3NF as academic rules) — useful conceptually, not something to memorize for its own sake.

Common beginner mistakes: - Designing tables without foreign keys, leading to orphaned/inconsistent data. - Not understanding the difference between `INNER JOIN` (only matches) and `LEFT JOIN` (all of left side). - Writing `SELECT *` in real queries instead of selecting only needed columns (a performance and clarity habit).

Free resources: "SQLBolt" (free, interactive), PostgreSQL official tutorial, "Mode Analytics SQL Tutorial" (free).

Estimated time: 12 hours.

Week 20 — Databases II: Prisma ORM & Schema Design

Build: Rebuild the Week 19 library system schema using Prisma, write the same 10 queries using Prisma's client, and compare the generated SQL to what you wrote by hand.

Capstone update: Define TaskForge AI's full schema in `schema.prisma` (Users, Projects, Tasks, ProjectMembers, with proper relations and enums for task status/priority). Run your first migration against the local Postgres database. Wire the Week 18 endpoints to read/write real data instead of in-memory arrays.

Learn: Prisma — the dominant TypeScript ORM in 2026 — and how to design a real, evolving schema with migrations.

Why: This is where TaskForge AI stops being a toy and becomes a real, persistent application. Prisma is heavily used at startups for its type-safety and DX.

Detailed topics: 1. **Why an ORM** - Tradeoffs: type safety and productivity vs. raw SQL control - When to drop to raw SQL even with an ORM (complex reporting queries) 2. **Prisma schema** - Models, fields, types, relations (`@relation`), enums - `@id`, `@default`, `@unique`, `@updatedAt` 3. **Migrations** - `prisma migrate dev`, understanding the generated SQL migration files - Migration history and why you never edit an already-applied migration in a shared environment 4. **Prisma Client** - CRUD operations (`create`, `findMany`, `findUnique`, `update`, `delete`) - `include`/`select` for relations - Filtering, pagination (`skip`/`take`), sorting 5. **Seeding** - Writing a seed script for consistent local dev data

Recommended learning order: Why ORM → schema design → migrations → Prisma Client CRUD → relations (`include`) → pagination/filtering → seeding.

Mandatory: All of the above. **Optional:** Prisma's raw query escape hatches (`$queryRaw`) — good to know exists, not needed yet.

Common beginner mistakes: - Designing the Prisma schema without first sketching the relationships on paper (leads to painful re-migrations). - Over-fetching relations with `include` everywhere instead of `select`ing only needed fields. - Not writing a seed script, making local development tedious and untestable.

Free resources: Official Prisma docs (excellent, free, has a full "Getting Started" track), Prisma's own YouTube "Prisma in 100 seconds"-style short intros for concepts.

Estimated time: 12 hours.

Week 21 — NoSQL & MongoDB (When and Why)

Build: A small "Activity Log" service using MongoDB directly (no ORM) — logging flexible, schema-varying events (different event types with different fields).

Capstone update: Add an `ActivityLog` / `AITaskSuggestion` collection in MongoDB for TaskForge AI (justified: this data is unstructured/varied and doesn't need relational integrity, unlike core Projects/Tasks). Document in `docs/spec.md` *why* this is a polyglot-persistence decision, not just "using two databases because you can."

Learn: Document databases and — more importantly — the judgment of when SQL vs. NoSQL is the right call, which is a very common system-design interview question.

Why: Real systems often use more than one database type. Understanding the actual trade-offs (not "NoSQL is faster," which is often false) is what separates junior from mid-level thinking.

Detailed topics: 1. **Document database concepts** - Collections, documents, flexible schemas - Embedding vs. referencing (the NoSQL equivalent of the join decision) 2. **MongoDB basics** - CRUD operations with the MongoDB driver (or Mongoose, awareness level) - Basic querying, filtering, indexes 3. **SQL vs. NoSQL decision framework** - Structured/relational data with integrity needs → SQL - Flexible, high-write, loosely-structured data → document DB - Real hybrid ("polyglot persistence") architectures at real companies 4. **CAP theorem (introductory — deep dive comes back in Week 32)**

Recommended learning order: Document concepts → MongoDB CRUD → embedding vs. referencing → SQL vs NoSQL decision framework → CAP intro.

Mandatory: Document concepts, basic CRUD, the decision framework. **Optional:** Mongoose ODM mastery (many teams use the native driver or Prisma's Mongo support instead in 2026 — awareness is enough).

Common beginner mistakes: - Choosing MongoDB for everything because it's "easier to start," then fighting for relational integrity later. - Embedding data that grows unbounded (e.g., embedding all comments inside a post document forever). - Not indexing frequently-queried fields, causing slow queries at even moderate scale.

Free resources: Official MongoDB docs + free MongoDB University courses, "SQL vs NoSQL" comparison articles from reputable engineering blogs (e.g., AWS's own database comparison docs).

Estimated time: 8–10 hours.

Week 22 — Authentication & Authorization

Build: A standalone "Auth Service" — signup, login, password hashing, JWT issuing, protected route middleware — as a focused mini project before wiring it into the capstone.

Capstone update: Implement real authentication for TaskForge AI: signup/login endpoints, hashed passwords, JWT-based sessions, and role-based authorization (e.g., project `owner` vs. `member` permissions on task edit/delete). Connect the frontend's login form (built in Week 13) to real endpoints.

Learn: How authentication and authorization actually work under the hood — one of the most commonly botched areas in junior developers' projects and a frequent interview topic.

Why: Nearly every real application needs auth. Getting it wrong is a security risk; understanding it deeply is a strong interview and portfolio signal.

Detailed topics: 1. **Authentication fundamentals** - Authentication vs. authorization (the distinction, precisely) - Password hashing with bcrypt (never storing plain text, why salting matters) 2. **Session-based vs. token-based auth** - Cookies + server sessions (stateful) - JWTs (stateless): structure (header/payload/signature), signing, expiry - Tradeoffs between the two approaches 3. **Implementing JWT auth** - Issuing access tokens (and refresh tokens — conceptual + basic implementation) - Storing tokens client-side safely (httpOnly cookies vs. localStorage tradeoffs and XSS risk) - Auth middleware to protect Express routes 4. **Authorization** - Role-based access control (RBAC) basics - Resource-level authorization (e.g., "only this project's members can edit its tasks") 5. **OAuth (conceptual + "Sign in with Google" basics)** - The OAuth 2.0 authorization code flow at a conceptual level - When to build your own auth vs. using a provider (Auth0, Clerk, Supabase Auth) — a real architecture decision worth understanding even if you implement your own for learning purposes

Recommended learning order: Auth vs. authz distinction → password hashing → session vs. JWT tradeoffs → implementing JWT auth end-to-end → authorization/RBAC → OAuth concept.

Mandatory: Password hashing, JWT implementation, auth middleware, basic RBAC. **Optional:** Building full OAuth flow from scratch (understanding it conceptually is enough; using a library/provider in real projects is normal and expected).

Common beginner mistakes: - Storing plain-text passwords (a genuinely disqualifying mistake in a real interview/portfolio review). - Storing JWTs in `localStorage` without understanding the XSS tradeoff vs. httpOnly cookies. - Not checking authorization on the backend at all — trusting frontend checks alone (a real, serious vulnerability class).

Free resources: "JWT.io" (free, has an excellent debugger + intro), OWASP Authentication Cheat Sheet, bcrypt npm package official docs.

Estimated time: 14 hours (this week deserves extra time — it's foundational and security-critical).

Week 23 — API Design Best Practices & Validation

Build: Add robust request validation (Zod) and consistent error handling to the Week 18 Bookstore API mini project. Write API documentation for it using a simple OpenAPI/Swagger setup.

Capstone update: Add Zod validation schemas to every TaskForge AI endpoint (reusing types shared with the frontend where possible). Standardize error responses across the whole API. Write basic API documentation (`docs/api.md` or a Swagger/OpenAPI spec).

Learn: Production-grade API hygiene: validation, consistent error shapes, and documentation — the difference between a "tutorial API" and a "hire-able API."

Why: Real APIs are consumed by teammates, other services, and sometimes external partners. Sloppy, undocumented, unvalidated APIs are a common junior-developer tell in code review.

Detailed topics: 1. **Input validation** - Why you never trust client input, ever - Zod schemas for request body/query/params validation - Sharing types between frontend and backend (a genuine architecture win of the TS + Zod combo) 2. **Consistent error handling** - A standard error response shape across the whole API - Centralized error-handling middleware (revisited from Week 18, now production-grade) - Custom error classes (`NotFoundError`, `ValidationError`, `UnauthorizedError`) 3. **API versioning (awareness level)** - Why/when APIs need versioning (`/api/v1/...`) 4. **REST vs. GraphQL (conceptual comparison, REST remains primary in this roadmap)** - What problems GraphQL solves, why this roadmap

focuses on REST (still the majority default at most startups), and what to read later if a job requires GraphQL 5. **API documentation** - Why documentation matters even for a "just me" project - Basic OpenAPI/Swagger spec, or a well-structured markdown API reference

Recommended learning order: Input validation → centralized error handling → custom error classes → API versioning (awareness) → REST vs GraphQL (awareness) → documentation.

Mandatory: Validation, centralized error handling, documentation. **Optional:** Full GraphQL implementation (this roadmap deliberately keeps you REST-focused since it's still the dominant standard; GraphQL is a valuable "next thing to learn" after you're employed).

Common beginner mistakes: - Trusting `req.body` without validating it, leading to crashes or corrupted data from malformed requests. - Returning inconsistent error shapes across different endpoints, making the API painful for any client to consume. - Never documenting the API, which becomes a real liability the moment anyone else (or future-you) needs to use it.

Free resources: Official Zod docs, OWASP Input Validation Cheat Sheet, Swagger/OpenAPI official "Getting Started" guide.

Estimated time: 10 hours.

Week 24 — Backend Architecture: Layers, Errors, Logging

Build: Refactor the Bookstore API into a proper layered architecture (routes → controllers → services → data access), and add structured logging.

Capstone update: Refactor TaskForge AI's backend into clean layers: routes handle HTTP only, controllers orchestrate, services hold business logic, and a data-access layer wraps Prisma calls. Add structured logging (request logging + error logging) using a library like `pino` or `winston`.

Learn: How to structure a backend so it stays maintainable as it grows — the architecture skill that separates "can follow a tutorial" from "can work on a real team's codebase."

Why: Every real backend job involves working in (and improving) an existing architecture. Understanding *why* layers exist — not just copying a folder structure — is what interviewers probe for in architecture questions.

Detailed topics: 1. **Layered architecture** - Routes (HTTP concerns only) → Controllers (request/response orchestration) → Services (business logic) → Data access (Prisma/DB calls) - Why each layer should be replaceable/testable independently 2. **Separation of concerns in practice** - What belongs in a controller vs. a service (a very concrete, practiceable skill) - Dependency direction (higher layers depend on lower layers, not vice versa) 3. **Error handling architecture** - Custom error hierarchy revisited, and how it flows cleanly through layers to the centralized handler 4. **Logging** - Structured (JSON) logging vs. `console.log` — why it matters for real production debugging - Log levels (`debug`, `info`, `warn`, `error`), request-scoped logging (correlation IDs, awareness level) 5. **Configuration management** - Centralizing env var access (one config module, not `process.env` scattered everywhere)

Recommended learning order: Layered architecture concept → controllers vs services distinction → dependency direction → error flow through layers → structured logging → centralized config.

Mandatory: All of the above. **Optional:** Correlation IDs / distributed tracing concepts (relevant more at microservices scale — good awareness, not needed for a single-service app).

Common beginner mistakes: - Putting Prisma calls directly in route handlers ("it works" but becomes unmaintainable and untestable). - Scattering `process.env.X` calls throughout the codebase instead of one config module. - Logging with `console.log` everywhere, then having no useful information when something breaks in production.

Free resources: "Node.js Design Patterns" (free chapter previews), various reputable engineering blog posts on "layered/clean architecture in Node," official `pino` docs.

Estimated time: 10 hours.

Week 25 — Backend Testing

Build: Write a full test suite for the Bookstore API: unit tests for services (mocking the data layer) and integration tests for endpoints (using Supertest against a real test database).

Capstone update: Write unit tests for TaskForge AI's service layer (task creation logic, authorization checks) and integration tests for the critical endpoints (auth, task CRUD). Set up a separate test database so tests never touch dev/prod data.

Learn: Backend testing patterns — unit vs. integration — and how to test code that touches a real database without it being slow or flaky.

Why: Backend tests are one of the strongest signals in a portfolio review, and testing is a daily activity on real engineering teams.

Detailed topics: 1. **Unit testing services** - Mocking the Prisma client / data layer so service logic is tested in isolation - Testing business logic edge cases (e.g., "can't assign a task to a non-member") 2. **Integration testing with Supertest** - Testing real HTTP requests against your Express app - Testing auth flows end-to-end (signup → login → access protected route) 3. **Test database strategy** - Separate test DB, resetting state between tests (transactions or truncation strategies) - Seeding test data reliably 4. **Test coverage (used wisely)** - What coverage numbers do and don't tell you - Prioritizing tests for business-critical paths over 100% coverage vanity metrics

Recommended learning order: Unit testing services (mocking) → integration testing with Supertest → test database strategy → coverage (as a tool, not a goal).

Mandatory: Service unit tests, endpoint integration tests, test DB isolation. **Optional:** Chasing 100% coverage (actively discouraged — prioritize critical paths).

Common beginner mistakes: - Running tests against the real dev database, causing flaky tests and data pollution. - Testing only the happy path and skipping error/edge cases (unauthorized access, invalid input, not-found resources). - Treating coverage percentage as the goal instead of a signal.

Free resources: Official Vitest/Jest docs (testing backend code), Supertest official docs, Kent C. Dodds' "Testing Trophy" article (free, changes how you think about test types).

Estimated time: 12 hours.

Week 26 — File Uploads, Third-Party APIs, Background Jobs

Build: Add avatar/file upload support to the Auth Service mini project (using a free-tier cloud storage like Cloudinary or S3-compatible storage), and integrate one free third-party API (e.g., an email-sending API like Resend's free tier) to send a "welcome email."

Capstone update: Add file upload support to TaskForge AI (task attachments or project cover images). Integrate a transactional email API for notifications (e.g., "you were assigned a task"). Introduce a very basic background job pattern (e.g., a simple queue or scheduled function) for something like "send a daily digest email" — awareness-level implementation, not a full job-processing system.

Learn: The "glue" skills that almost every real product needs — handling files, talking to third-party services, and doing work outside the request/response cycle.

Why: Nearly every real-world app uploads files and integrates third-party services. These are extremely common, practical, high-value skills rarely covered well in beginner tutorials.

Detailed topics: 1. **File uploads** - `multipart/form-data`, handling uploads with `multer` (or equivalent) - Uploading to cloud storage (S3-compatible or Cloudinary) instead of the local filesystem (why local storage doesn't work in most deployment environments) - Validating file type/size, basic security considerations (never trusting file extensions alone) 2. **Integrating third-party APIs** - Reading API docs effectively, API keys and secret management - Rate limits, retries, timeout handling - Webhooks (conceptual): what they are, how to receive and verify one 3. **Background jobs / async work (introductory)** - Why some work shouldn't block the request/response cycle - Simple approaches: `setImmediate`/cron-style scheduled functions, vs. a real queue (BullMQ, awareness level) for when scale demands it 4. **Sending transactional email** - Templating basics, using a provider's free tier

Recommended learning order: File uploads (local handling → cloud storage) → third-party API integration patterns → webhooks (concept) → background jobs (concept + simple implementation) → transactional email.

Mandatory: File upload to cloud storage, one real third-party API integration, understanding of why background jobs exist. **Optional:** A full production-grade queue system like BullMQ (valuable to know exists; overkill to master for this project's scale).

Common beginner mistakes: - Saving uploaded files to local disk in a way that breaks the moment the app is deployed (most hosting platforms have ephemeral filesystems). - Hardcoding API keys in source code instead of environment variables. - Doing slow work (like sending emails) synchronously inside the request handler, making users wait unnecessarily.

Free resources: `multer` official docs, AWS S3 free-tier docs or Cloudinary free docs, Resend (or similar) official docs, "Webhooks explained" free articles from Stripe's or similar reputable engineering blogs.

Estimated time: 10–12 hours.

Phase 3 milestone check: TaskForge AI is now a genuinely full-stack, persisted, authenticated, tested application — frontend and backend fully wired together. This is already a strong portfolio piece even before AI features and deployment.

Phase 4 — Software Engineering Craft (Weeks 27–31)

Week 27 — Clean Code Principles

Build: Take a deliberately messy 200-line script (write one yourself, badly, on purpose — bad names, deep nesting, duplicated logic) and refactor it into clean code, documenting each change and why you made it.

Capstone update: Do a full "clean code audit" pass over TaskForge AI's backend service layer: rename unclear variables/functions, extract long functions, remove duplication, reduce nesting.

Learn: The principles that make code readable and maintainable by other humans (including future-you) — not clever code, clear code.

Why: Code review is where junior engineers are evaluated most closely. "Clean code" isn't dogma — it's a practical skill for surviving and thriving in a real codebase with teammates.

Detailed topics: 1. **Naming** - Intention-revealing names, avoiding abbreviations/ambiguity - Function names as verbs, variable names as nouns, boolean names as questions (`isActive`, `hasPermission`) 2. **Functions** - Single Responsibility Principle at the function level - Small functions, few parameters, avoiding side effects where possible - Avoiding deep nesting (early returns/guard clauses) 3. **Comments** - When comments are necessary (the "why," not the "what") vs. when they signal the code itself should be clearer 4. **DRY (Don't Repeat Yourself) — and its limits** - Real duplication vs. coincidental similarity (over-abstracting too early is also a real anti-pattern) 5. **Code smells** - Long functions, large classes/files, deep nesting, magic numbers/strings, shotgun surgery (a change requiring edits in many unrelated places)

Recommended learning order: Naming → functions/SRP → nesting/guard clauses → comments → DRY and its limits → code smells as a checklist.

Mandatory: All of the above. **Optional:** N/A — this is foundational craft, revisit throughout the rest of the roadmap rather than skip any part.

Common beginner mistakes: - Over-abstracting too early ("DRY-ing" two superficially similar but conceptually unrelated pieces of code into a confusing shared function). - Writing comments that restate the code instead of explaining non-obvious reasoning. - Treating "clean code" as a one-time pass instead of an ongoing habit during regular development.

Free resources: "Clean Code" concepts summarized in many free blog posts/talks (the book itself isn't free, but its core ideas are widely covered for free — search "Clean Code summary" from reputable engineering blogs), Refactoring.Guru's free "Code Smells" catalog.

Estimated time: 8 hours.

Week 28 — Design Patterns

Build: Implement 5 design patterns from scratch in small, isolated TypeScript examples: Singleton, Factory, Strategy, Observer, and Repository — each with a realistic mini use case (not abstract shapes/animals examples).

Capstone update: Identify and apply at least 2 patterns genuinely useful in TaskForge AI — e.g., a Repository pattern for data access (formalizing what you built in Week 24), and a Strategy pattern for, say, different task-sorting algorithms or notification channels (email vs. in-app).

Learn: The most practically useful design patterns for JS/TS applications — not the full Gang of Four catalog, just the ones that show up constantly in real code.

Why: Design patterns give you shared vocabulary with other engineers and solve recurring problems elegantly. Misusing or over-applying them is also a well-known anti-pattern, so judgment matters as much as knowledge.

Detailed topics: 1. **Creational patterns** - Factory pattern (creating objects without specifying exact classes) - Singleton (and why it's often overused/misused — e.g., a single Prisma client instance is a legitimate use) 2. **Structural/behavioral patterns most relevant to JS/TS apps** - Repository pattern (abstracting data access — you already used this informally in Week 24) - Strategy pattern (interchangeable algorithms/behaviors behind a common interface) - Observer pattern (the conceptual backbone of event emitters, pub/sub, and even React's re-rendering model) 3. **When NOT to use a pattern** - Recognizing when a pattern adds ceremony without solving a real problem 4. **Dependency Injection (conceptual)** - Why decoupling construction from usage makes testing easier (ties directly back to Week 25's service mocking)

Recommended learning order: Factory → Singleton (and its risks) → Repository → Strategy → Observer → dependency injection concept → judgment on when to skip patterns.

Mandatory: Factory, Repository, Strategy, Observer, DI concept. **Optional:** The full Gang of Four catalog (23 patterns) — most rarely appear in modern JS/TS app code; this roadmap deliberately focuses on the practically common ones.

Common beginner mistakes: - Applying a pattern because it's "correct" rather than because it solves an actual problem in this codebase (overengineering). - Confusing Singleton-as-a-pattern with "just using global mutable state" (they're not the same, and the latter causes real bugs). - Not recognizing patterns you're already using informally (e.g., React's Context is essentially dependency injection).

Free resources: Refactoring.Guru (excellent free pattern catalog with TypeScript-adjacent examples), "Patterns.dev" (free, modern, JS-focused design patterns site).

Estimated time: 10 hours.

Week 29 — Software Architecture Fundamentals

Build: Write an "Architecture Decision Record" (ADR) — a real practice used at serious engineering companies — comparing monolith vs. microservices for a hypothetical scaling scenario, and justify a recommendation.

Capstone update: Write an ADR for TaskForge AI: "Why TaskForge AI is a modular monolith, not microservices, at this stage" — reasoning through team size, complexity, and deployment tradeoffs like a real engineer would, not just following a trend.

Learn: How to reason about system architecture at a level beyond "which framework do I use" — the kind of thinking system design interviews actually test.

Why: Architecture judgment is what separates junior from mid-level engineers. Being able to explain *why* you made a structural decision — not just that you made one — is a strong interview signal.

Detailed topics: 1. **Monolith vs. microservices** - What each actually means (a monolith isn't automatically "bad") - Tradeoffs: deployment complexity, team coordination, data consistency, operational overhead - Why most startups should start with a (well-structured, "modular") monolith 2. **Layered / hexagonal (ports and adapters) architecture (conceptual)** - The core idea: business logic shouldn't depend on frameworks/infrastructure details - How this connects to what you already built in Week 24 3. **Coupling & cohesion** - Why high cohesion + low coupling is the underlying goal of most architecture advice 4. **Architecture**

Decision Records (ADRs) - Why documenting *why* a decision was made (not just what) is valuable to a team and to your own future self 5. **Scaling a codebase over time (conceptual)** - Signs it's time to extract a service, split a monorepo, etc. — awareness level, revisited more concretely in Phase 5's system design weeks

Recommended learning order: Monolith vs. microservices tradeoffs → coupling/cohesion → layered/hexagonal concept → writing an ADR → signs of when to evolve architecture.

Mandatory: Monolith vs. microservices tradeoffs, coupling/cohesion, writing one real ADR. **Optional:** Full hexagonal architecture implementation (valuable conceptual exposure now; a from-scratch strict implementation is a deeper topic for later career growth).

Common beginner mistakes: - Reaching for microservices on a solo/small project because it's trendy, adding massive operational overhead for no real benefit. - Making architecture decisions and never writing down the reasoning, so nobody (including future-you) knows why. - Confusing "more files/folders" with "good architecture" — structure without clear boundaries doesn't actually help.

Free resources: Martin Fowler's free articles ("Monolith First," "Microservices" overview) on martinfowler.com, "Architecture Decision Records" (adr.github.io, free), free ThoughtWorks Technology Radar articles.

Estimated time: 8 hours.

Week 30 — Networking Fundamentals

Build: A written + diagrammed walkthrough (your own notes, not copied) of exactly what happens between typing `taskforge.ai` in a browser and seeing the page — DNS, TCP, TLS, HTTP request/response — annotated with real terms.

Capstone update: No new capstone code this week — instead, do a deep technical review of TaskForge AI's actual network behavior: inspect real requests in the browser's Network tab, verify HTTPS is enforced (in prep for deployment), and document the request/response flow for one full user action (e.g., "create a task") end to end.

Learn: The networking fundamentals that underlie every web app — genuinely useful for debugging, and one of the most common "so, how does the internet actually work" interview questions.

Why: Understanding what's happening below your framework is what lets you debug real production issues (CORS errors, latency, SSL problems) instead of guessing.

Detailed topics: 1. **DNS** - How domain name resolution works (conceptual, not deep protocol internals) 2. **TCP/IP basics** - What a "connection" actually is, the three-way handshake (conceptual) - TCP vs. UDP (awareness level — when each is used) 3. **HTTP/HTTPS** - Request/response structure: headers, methods, status codes (deepened from Week 18) - HTTP/1.1 vs HTTP/2 (awareness), keep-alive - TLS/SSL at a conceptual level — what "HTTPS" actually guarantees 4. **CORS** - What it actually is and why browsers enforce it (a very common real-world debugging topic) - Same-origin policy, preflight requests 5. **REST semantics revisited through a networking lens** - Idempotency (why `PUT`/`DELETE` should be idempotent, `POST` shouldn't be assumed to be) - Caching headers (conceptual level: `Cache-Control`, ETags)

Recommended learning order: DNS → TCP/IP basics → HTTP/HTTPS → CORS → idempotency/caching headers.

Mandatory: DNS concept, HTTP/HTTPS deep understanding, CORS, idempotency. **Optional:** TCP handshake/packet-level internals (good intellectual curiosity, not required for employability).

Common beginner mistakes: - Treating CORS errors as "the backend is broken" instead of understanding what CORS actually checks. - Not knowing the difference between 401 (unauthenticated) and 403 (unauthorized) — a very common interview trip-up. - Assuming HTTPS alone means an application is "secure" (it only secures data in transit).

Free resources: "How DNS Works" (free comic-style explainer, howdns.works), MDN's HTTP and CORS guides (excellent and thorough), Cloudflare's free "Learning Center" articles on networking basics.

Estimated time: 8 hours.

Week 31 — System Design Fundamentals I

Build: A written system design for a URL shortener (a classic, well-scoped practice problem): requirements gathering, high-level architecture diagram, data model, and a discussion of one bottleneck and how you'd address it.

Capstone update: Write a "TaskForge AI at 100,000 users" system design doc — a forward-looking, hypothetical exercise: what would break first (probably the database), and what would you change (read replicas? caching layer? Not implemented — just reasoned through, like a real interview answer).

Learn: The foundational vocabulary and thought process of system design: scalability, caching, and load balancing — enough to hold your own in an entry/mid-level system design interview.

Why: System design interviews are standard even for many junior/mid roles in 2026. This is also just genuinely useful engineering judgment.

Detailed topics: 1. **Scalability basics** - Vertical vs. horizontal scaling - Stateless vs. stateful services (why statelessness makes horizontal scaling easier) 2. **Load balancing** - What a load balancer does, common algorithms (round robin, least connections — conceptual) 3. **Caching** - Why caching helps, where it fits (client, CDN, server, database query cache) - Cache invalidation basics (the famously "hard problem") - In-memory caches (Redis) — conceptual introduction, revisited more in Week 32 4. **The system design interview framework** - Clarify requirements → estimate scale → high-level design → deep dive on one component → discuss tradeoffs - Practicing "think out loud" reasoning, not memorized answers

Recommended learning order: Scalability concepts → load balancing → caching → the interview framework, practiced on a real example problem.

Mandatory: Scalability concepts, caching, the interview framework. **Optional:** Deep load balancer algorithm internals (conceptual awareness is enough at this stage).

Common beginner mistakes: - Jumping straight to a "solution" in a system design discussion without clarifying requirements/scale first. - Treating caching as a free win without considering staleness/invalidation tradeoffs. - Over-designing a simple problem with unnecessary complexity ("resume-driven design").

Free resources: "System Design Primer" (free, extremely popular GitHub repo), Byte-ByteGo's free YouTube explainers, Gaurav Sen's free system design videos.

Estimated time: 10 hours.

Phase 4 milestone check: You should now be able to explain and defend architectural decisions in TaskForge AI — not just describe what you built, but why — which is exactly what technical interviews probe for.

Phase 5 — System Design, Security, Performance (Weeks 32–34)

Week 32 — System Design Fundamentals II: Data at Scale

Build: Extend the Week 31 URL shortener design: add a caching layer (Redis, conceptually designed + a small hands-on Redis exercise), discuss database sharding/replication at a conceptual level, and design for "what if this needs to handle 10x traffic."

Capstone update: Add a real Redis cache to TaskForge AI for one genuinely cacheable thing (e.g., project member lists, or rate-limiting login attempts) — hands-on, not just theoretical this time.

Learn: Database scaling concepts (replication, sharding), message queues, and the CAP theorem — the deeper system design vocabulary expected at mid-level interviews.

Why: These are the concepts that show up in almost every system design interview beyond the most junior level, and they matter for real production systems, too.

Detailed topics: 1. **Database scaling** - Read replicas (why/how they help read-heavy workloads) - Sharding/partitioning (conceptual — splitting data across databases) - Connection pooling (practical — you've likely already hit this with Prisma) 2. **CAP theorem (revisited from Week 21, now in depth)** - Consistency, Availability, Partition tolerance — why you can't have all three during a partition - How this maps to real database choices (Postgres favors consistency; some NoSQL systems favor availability) 3. **Message queues (conceptual)** - Why decoupling services with a queue helps (producer/consumer pattern) - Common tools at a name-recognition level (RabbitMQ, Kafka, SQS) — you don't need to operate these yet, just understand what problem they solve 4. **Redis in practice** - Key-value basics, `SET/GET/EXPIRE` - Common use cases: caching, session storage, rate limiting

Recommended learning order: Database scaling concepts → CAP theorem in depth → message queues (concept) → hands-on Redis.

Mandatory: Database scaling concepts, CAP theorem, hands-on Redis basics. **Optional:** Operating Kafka/RabbitMQ yourself (name-recognition + conceptual understanding is sufficient for this stage of your career).

Common beginner mistakes: - Reaching for sharding/microservices/queues on a small project "because big companies do it" — this is over-engineering for the actual scale. - Not understanding that read replicas introduce eventual consistency, which can cause subtle bugs if the app assumes instant consistency. - Using Redis as a primary data store without understanding its persistence tradeoffs.

Free resources: "System Design Primer" (continued), Redis official docs + free interactive tutorial, ByteByteGo free content on database scaling.

Estimated time: 10 hours.

Week 33 — Security Fundamentals

Build: Take the Bookstore API from earlier and deliberately find/fix 5 security issues (SQL injection risk if using raw queries, missing input validation, missing rate limiting, verbose error messages leaking stack traces, missing security headers).

Capstone update: Run a security pass on TaskForge AI: add rate limiting on auth endpoints, ensure error responses never leak stack traces/internal details in production, add `helmet`

security headers (if not already from Week 18), and verify authorization checks exist on every mutating endpoint (revisited from Week 22).

Learn: The OWASP Top 10 and practical secure-coding habits — genuinely important, and a topic technical interviewers increasingly probe for.

Why: Security mistakes are costly and common in junior developers' code. Demonstrating security awareness in a portfolio project is a strong, differentiating signal.

Detailed topics: 1. **OWASP Top 10 (the ones most relevant to a JS/TS full-stack app)** - Injection (SQL injection — and why parameterized queries/ORMs mostly protect you, but not always) - Broken authentication (revisit Week 22's mistakes list) - Sensitive data exposure (never logging passwords/tokens, encrypting sensitive fields when needed) - Cross-Site Scripting (XSS) — how it happens, why React helps by default, where it can still slip in (`dangerouslySetInnerHTML`) - Cross-Site Request Forgery (CSRF) — conceptual understanding, especially relevant with cookie-based auth - Security misconfiguration (default credentials, verbose error messages, missing headers) 2. **Secure coding habits** - Rate limiting (protecting login/signup endpoints from brute force) - Never trusting client-side validation alone (tie back to Week 23) - Dependency security (`npm audit`, keeping packages updated, awareness of supply-chain risk) 3. **Secrets management** - Environment variables, never committing secrets, rotating credentials if leaked 4. **HTTPS/TLS in practice (tie back to Week 30)**

Recommended learning order: OWASP Top 10 walkthrough (mapped to your own project) → secure coding habits → rate limiting hands-on → secrets management → dependency security.

Mandatory: OWASP Top 10 awareness applied to your project, rate limiting, secrets management, XSS/CSRF conceptual understanding. **Optional:** Deep penetration-testing skills (a specialization of its own — awareness is enough for a software engineering role).

Common beginner mistakes: - Believing "I used an ORM, so I'm safe from injection" without understanding when raw queries can still be vulnerable. - Leaving verbose error messages/stack traces exposed in production responses. - Never running `npm audit` or updating dependencies, leaving known vulnerabilities unpatched.

Free resources: OWASP Top 10 (official, free, the industry-standard reference), OWASP Cheat Sheet Series (free, extremely practical), `npm audit` official docs.

Estimated time: 10 hours.

Week 34 — Performance Optimization & Technical Documentation

Build: Run a full performance audit on the Kanban Board (frontend) mini project using Lighthouse, and fix at least 3 real issues found (bundle size, image optimization, unnecessary re-renders).

Capstone update: Run a full Lighthouse + backend performance pass on TaskForge AI: measure and improve frontend load performance, add database indexes where query patterns demand it (tie back to Week 19–20), and write a complete `README.md` — the kind that makes a recruiter or engineer immediately understand the project (setup instructions, architecture overview, screenshots, tech stack, what you learned).

Learn: How to measure and improve performance with real tools (not guesswork), and how to write technical documentation that makes your work legible to others.

Why: "It works on my machine and I never measured it" is a common junior gap. A great README is often the *first* thing anyone evaluating your portfolio actually reads.

Detailed topics: 1. **Frontend performance** - Lighthouse / Web Vitals (LCP, CLS, INP) — what they measure and why they matter - Bundle size analysis, code splitting (revisit Week 15), image optimization/lazy loading 2. **Backend performance** - Identifying slow queries (`EXPLAIN ANALYZE` in Postgres, conceptual + basic hands-on) - Adding indexes based on real query patterns (not guessing) - N+1 query problems — what they are, how Prisma's `include` can accidentally cause them, how to fix with proper eager-loading 3. **Technical documentation** - Writing an effective README (setup, architecture, decisions, screenshots/demo link) - Inline documentation vs. separate docs (when each is appropriate) - Writing for your audience: a recruiter skimming vs. an engineer evaluating code

Recommended learning order: Frontend performance auditing (Lighthouse) → bundle/code-splitting fixes → backend query performance (EXPLAIN ANALYZE, indexes) → N+1 detection/fixes → README/documentation writing.

Mandatory: Lighthouse audit + fixes, N+1 query awareness, a genuinely strong README. **Optional:** Deep database query-planning expertise (useful to revisit later in your career as data volume grows).

Common beginner mistakes: - Never running a performance audit tool and assuming the app is "fast enough" without evidence. - Accidentally causing N+1 queries with ORM relation loading and not noticing until the app is slow. - Shipping a project with no README, forcing anyone reviewing it to reverse-engineer what it even does.

Free resources: Google's official Web Vitals docs + Lighthouse (built into Chrome DevTools, free), PostgreSQL official docs on `EXPLAIN`, "Make a README" (makeareadme.com, free guide).

Estimated time: 10 hours.

Phase 5 milestone check: TaskForge AI is now secure, reasonably performant, and properly documented — the engineering-craft layer that turns "a project that works" into "a project that reflects real engineering judgment."

Phase 6 — Cloud, Deployment, CI/CD, Docker (Weeks 35–38)

Week 35 — Cloud Fundamentals

Build: Deploy the standalone Bookstore API (a low-stakes practice target) to a free-tier cloud platform end to end, including environment variables and a managed database.

Capstone update: Choose and provision real infrastructure for TaskForge AI: a managed Postgres instance (e.g., Neon, Supabase, or Railway's free tier), and set up environment configs for `development` vs. `production`.

Learn: Core cloud concepts — enough to deploy and reason about real infrastructure, without needing to become a dedicated DevOps engineer.

Why: "It runs on localhost" isn't a finished project. Deployment is where a huge share of self-taught developers' projects quietly die — this phase makes sure yours doesn't.

Detailed topics: 1. **Cloud fundamentals** - IaaS vs. PaaS vs. serverless (conceptual — where do Vercel/Railway/Render/AWS each fit) - Managed databases vs. self-hosting (why managed is the right default for a solo/small project) 2. **Environment configuration** - Environment-specific configs (dev/staging/prod), 12-Factor App principles (awareness level — the core ideas, not the full manifesto memorized) - Secrets management in cloud platforms (never hardcoding, using the platform's secret store) 3. **DNS & domains (practical)** - Buying/connecting a custom domain, DNS records (A, CNAME) at a practical level 4. **Basic AWS awareness (name-recognition level, since it's still the industry default many jobs assume)** - S3 (object storage), EC2 (VMs), RDS (managed databases) — what they are, not deep hands-on mastery required for this roadmap

Recommended learning order: IaaS/PaaS/serverless concepts → environment configuration → managed database provisioning → DNS/domains → AWS awareness pass.

Mandatory: PaaS-level deployment understanding, environment configuration, managed database setup. **Optional:** Deep AWS hands-on (EC2/VPC/IAM mastery) — valuable for later career growth, not required to get your first role in 2026's PaaS-friendly startup landscape.

Common beginner mistakes: - Hardcoding `localhost` URLs/ports that break the moment the app is deployed. - Committing production secrets into the repo "just to get it working," then forgetting to remove them. - Choosing a self-hosted database on day one when a managed free-tier option would save weeks of operational headache.

Free resources: Railway/Render/Vercel official docs (all have generous free tiers and excellent guides), "The Twelve-Factor App" (12factor.net, free), freeCodeCamp's cloud/deployment articles.

Estimated time: 8 hours.

Week 36 — Deployment: Frontend, Backend, Database

Build: Fully deploy the standalone Auth Service + a tiny frontend for it, end to end, with a real custom subdomain and HTTPS.

Capstone update: Fully deploy TaskForge AI: frontend (Vercel or Netlify), backend (Railway/Render/Fly.io), database (managed Postgres from Week 35), with environment variables configured correctly in each platform, CORS configured for the real production domains, and HTTPS enforced everywhere. This is a major milestone — a real, live, shareable URL.

Learn: The end-to-end mechanics of shipping a full-stack TypeScript app to production.

Why: This is the moment your capstone becomes something you can put in a resume/portfolio as a live link, not just a GitHub repo — a massive credibility jump for interviews.

Detailed topics: 1. **Frontend deployment** - Build process (`vite build`), static hosting on Vercel/Netlify - Environment variables at build time vs. runtime (a common source of confusion) 2. **Backend deployment** - Deploying a Node/Express app (Railway, Render, Fly.io, or similar) - Health check endpoints, process management basics (why a process manager/restart policy matters) 3. **Database connection in production** - Connection strings, connection pooling in a serverless/PaaS context (a genuinely common gotcha with Prisma + serverless) - Running migrations against production safely 4. **CORS & domains in production** - Correctly configuring CORS for real frontend/backend domains (revisit Week 30) - Custom domain + HTTPS setup on your chosen platforms 5. **Rollbacks & deployment safety (conceptual)** - Why you always want an easy way to roll back a bad deploy

Recommended learning order: Frontend deploy → backend deploy → production database connection → CORS/domain config → rollback awareness.

Mandatory: All of the above — this week must end with a genuinely live, working app. **Optional:** Blue-green/canary deployment strategies (valuable awareness for later, not needed at this project's scale).

Common beginner mistakes: - Forgetting to set production environment variables on the hosting platform (works locally, breaks live). - CORS misconfiguration between the deployed frontend and backend domains. - Running a destructive migration directly against production without a backup/rollback plan.

Free resources: Vercel/Netlify official deployment docs, Railway/Render official docs, Prisma's official "Deploying to production" guide.

Estimated time: 12 hours.

Week 37 — CI/CD with GitHub Actions

Build: Set up a GitHub Actions pipeline for the Bookstore API: run lint + tests automatically on every pull request, and block merges if they fail.

Capstone update: Set up a full CI/CD pipeline for TaskForge AI: on every push, automatically run linting, type-checking, and the full test suite; on merge to `main`, automatically deploy to production (or trigger the hosting platform's auto-deploy correctly configured with these gates).

Learn: Continuous Integration/Continuous Deployment — how real engineering teams ship code safely and frequently.

Why: CI/CD is standard practice at essentially every serious engineering team in 2026. Being able to say "my project has a real CI pipeline" is a concrete, checkable claim in an interview.

Detailed topics: 1. **CI fundamentals** - What CI actually solves (catching problems before merge, not after) - GitHub Actions basics: workflows, jobs, steps, triggers (`on: push`, `on: pull_request`) 2. **Building a real pipeline** - Installing dependencies, caching for speed - Running lint, type-check, and test steps - Failing the build correctly (exit codes) so broken code can't merge 3. **CD fundamentals** - Automatic deployment on merge to `main` (or a manual-approval gate for production — conceptual awareness of both models) - Environment secrets in GitHub Actions (`secrets` context) 4. **Branch protection** - Requiring passing checks before merge, basic branch protection rules on GitHub

Recommended learning order: CI fundamentals → building the workflow file → lint/type-check/test steps → CD trigger on merge → branch protection rules.

Mandatory: All of the above. **Optional:** More advanced pipeline strategies (matrix builds, multi-environment deploy gates) — valuable to explore later as needed.

Common beginner mistakes: - Writing a CI pipeline that doesn't actually fail the build on errors (a false sense of safety). - Committing secrets directly into workflow files instead of using GitHub's encrypted secrets. - Never enabling branch protection, so the CI checks exist but don't actually block bad merges.

Free resources: Official GitHub Actions docs (thorough and free), GitHub's own "Learn GitHub Actions" free guide.

Estimated time: 8 hours.

Week 38 — Docker Fundamentals

Build: Containerize the Bookstore API (Dockerfile + docker-compose with a Postgres service) and run the entire stack with a single `docker compose up`.

Capstone update: Write a `Dockerfile` for TaskForge AI's backend and a `docker-compose.yml` that spins up the API + Postgres + Redis locally in one command — dramatically simplifying onboarding (a real engineering-team concern) even for a solo project.

Learn: Containerization fundamentals — genuinely useful cross-platform (works the same on your Windows machine as it will in production), and a very common requirement in job postings.

Why: Docker knowledge is expected at most product companies by the time you're past entry-level, and it solves the classic "works on my machine" problem you'll otherwise keep running into.

Detailed topics: 1. **Container fundamentals (conceptual)** - Containers vs. virtual machines (why containers are lighter weight) - Images vs. containers (the class vs. instance analogy) 2. **Docker on Windows** - Docker Desktop setup, WSL2 backend (used transparently by Docker Desktop — you don't need to work in Linux directly, Docker handles this) 3. **Writing a Dockerfile** - `FROM`, `WORKDIR`, `COPY`, `RUN`, `CMD/ENTRYPOINT` - Multi-stage builds (conceptual + basic hands-on — smaller, production-ready images) - `.dockerignore` 4. **Docker Compose** - Defining multi-service apps (API + Postgres + Redis) in one `docker-compose.yml` - Networking between containers, volumes for persistent data 5. **Basic image management** - `docker build`, `docker run`, `docker ps`, `docker logs`, cleaning up unused images/containers

Recommended learning order: Container concepts → Docker Desktop setup → writing a basic Dockerfile → multi-stage builds → Docker Compose for multi-service local dev → image management basics.

Mandatory: All of the above. **Optional:** Kubernetes (explicitly out of scope for this roadmap — it solves problems at a scale you don't have yet; worth learning only once a job requires it).

Common beginner mistakes: - Not using a `.dockerignore`, resulting in bloated images that include `node_modules` or `.git`. - Writing a single-stage Dockerfile that ships dev dependencies and build tools into production, creating unnecessarily large/insecure images. - Not understanding container networking, leading to confusing "connection refused" errors between services in Compose.

Free resources: Official Docker docs + "Get Started" guide, Docker Desktop for Windows official setup guide, "Docker Compose" official docs.

Estimated time: 10 hours.

Phase 6 milestone check: TaskForge AI is now live in production, backed by real CI/CD, and fully containerized for local development — this is the exact operational maturity level real engineering teams expect.

Phase 7 — AI Engineering for Software Engineers (Weeks 39–43)

Framing for this phase: You explicitly don't want to become an AI researcher — this phase is scoped accordingly. The goal is to become a software engineer who uses AI tools expertly and can ship AI-powered features competently, which is now a genuinely differentiating and in-demand skill set.

Week 39 — AI-Assisted Software Development & Prompt Engineering for Developers

Build: Deliberately re-do one earlier mini project (e.g., the Kanban Board) using an AI coding assistant (Claude Code, Cursor, or GitHub Copilot) as a full workflow partner — but write a reflection doc comparing what the AI got right, what it got subtly wrong, and how you caught the mistakes.

Capstone update: Retroactively review TaskForge AI's codebase with an AI coding assistant: ask it to identify potential bugs, suggest refactors, and flag security issues — then critically evaluate every suggestion before applying anything (never accept-and-forget).

Learn: How to use AI coding tools as a force multiplier without becoming dependent on them or shipping code you don't understand.

Why: This is the single most impactful modern skill this roadmap can add. Every hiring manager in 2026 expects engineers to use AI tools effectively — but also expects them to still deeply understand their own code.

Detailed topics: 1. **The AI-assisted development landscape** - Chat-based assistants vs. IDE-integrated tools (Copilot, Cursor) vs. agentic coding tools (Claude Code) — what each is actually good at 2. **Prompt engineering for developers specifically** - Being explicit about context, constraints, and desired output format - Providing relevant code/file context instead of vague requests - Iterative prompting (refining instead of re-explaining from scratch) - Asking for explanations, not just code, to preserve your own understanding 3. **Using AI without losing skill** - The "read every line before accepting" discipline - Using AI for boilerplate/repetitive work, doing the architectural thinking yourself - Recognizing when AI-generated code is subtly wrong (plausible-looking but incorrect logic, outdated APIs, security gaps) 4. **Debugging AI-generated code** - Treating AI output as a first draft from a junior collaborator, not ground truth - Common AI-generated-code failure patterns (hallucinated library methods, ignoring edge cases, inconsistent error handling) 5. **Using AI responsibly** - Not pasting secrets/proprietary code into public tools without checking data-handling policies - Being transparent (in interviews and on teams) about AI usage rather than hiding it

Recommended learning order: Landscape overview → prompt engineering fundamentals → hands-on with your chosen tool → the "read everything" discipline → debugging AI output → responsible use practices.

Mandatory: Hands-on fluency with at least one AI coding assistant, the "never blindly accept" discipline, responsible-use awareness. **Optional:** Mastering every available AI coding tool (pick one or two well rather than shallowly trying everything).

Common beginner mistakes: - Accepting AI-generated code without reading/understanding it, then being unable to explain your own project in an interview. - Vague prompts ("fix my code") instead of specific, context-rich prompts, leading to poor suggestions. - Pasting sensitive data/secrets into a chat tool without checking its data policy.

Free resources: Anthropic's own prompt engineering documentation (docs.claude.com), GitHub Copilot's official "Best practices" docs, Claude Code's official documentation.

Estimated time: 10 hours.

Week 40 — LLM Fundamentals & AI APIs

Build: A small standalone Node/TS script that calls an LLM API directly (no framework) to summarize a block of text, then extend it to support streaming responses and structured JSON output.

Capstone update: Design (on paper) the specific AI features TaskForge AI will get in the coming weeks: "AI task breakdown" (turn a vague task description into subtasks) and "AI project summary" (summarize recent activity). Write the exact API contracts for these new endpoints now, before building.

Learn: How LLMs actually work at a practical (not research) level, and how to integrate an LLM API into a real application.

Why: "Building AI-powered applications" is explicitly on your goal list. This requires understanding tokens, context windows, and API mechanics — not deep ML theory.

Detailed topics: 1. **LLM fundamentals (practitioner level, not research level)** - What a token is, how context windows work and why they're a real engineering constraint - Temperature and other sampling parameters (practical effect, not the underlying math) - Why LLMs "hallucinate" and what that means for how you design features around them 2. **Working with AI APIs** - The `/messages`-style request/response shape (system prompt, user/assistant turns) - API keys and secure server-side usage (never calling LLM APIs directly from the frontend with an exposed key) - Streaming responses (why and how, and the UX benefit) - Structured/JSON outputs (getting reliable, parseable data back from an LLM for real app logic) 3. **Function calling / tool use (conceptual + basic hands-on)** - What it is, why it lets an LLM take real actions in your app 4. **Cost & rate-limit awareness** - Token-based pricing, why prompt/response length has a real cost implication - Basic rate limiting/retry handling for API calls

Recommended learning order: LLM fundamentals (tokens/context) → basic API call → streaming → structured outputs → function calling concept → cost/rate-limit awareness.

Mandatory: All of the above except deep function-calling architecture (introduced here, built on in Week 43). **Optional:** Fine-tuning models (out of scope — not needed for "software engineer who builds with AI," and rarely the right tool compared to good prompting/RAG for most product use cases).

Common beginner mistakes: - Calling an LLM API directly from frontend JavaScript, exposing the API key publicly. - Assuming LLM output is always correct/reliable without validating structured outputs before using them in app logic. - Ignoring cost implications and sending unnecessarily large context on every request.

Free resources: Anthropic's official API documentation (docs.claude.com) — includes free guides on prompting, streaming, and tool use, OpenAI's API documentation for comparison/breadth.

Estimated time: 10 hours.

Week 41 — Building AI-Powered Features

Build: A standalone "AI Text Summarizer" web app (React frontend + Node backend) with a real chat-style streaming UI — a focused project purely about the AI-integration UX pattern.

Capstone update: Implement the first real AI feature in TaskForge AI: "AI task breakdown" — a user describes a task in plain language, and the backend calls an LLM API to return structured subtask suggestions (using the structured-output pattern from Week 40), which the user can accept/edit/reject before saving.

Learn: The full-stack pattern for shipping AI features in a real product: backend proxying, streaming to the frontend, and designing UX around non-deterministic output.

Why: This is the concrete, portfolio-visible proof that you can "build AI-powered applications," not just talk about LLMs abstractly.

Detailed topics: 1. **Backend patterns for AI features** - Never exposing LLM API keys to the client — the backend proxies every AI call - Structuring an "AI service" layer, consistent with your existing layered architecture (Week 24) - Handling and validating structured LLM output before trusting it in your database 2. **Streaming to the frontend** - Server-Sent Events (SSE) or streaming fetch responses for a real-time "typing" UI feel - Handling partial/incomplete data gracefully on the frontend 3. **Designing UX for non-deterministic AI output** - Always giving the user a way to review/edit/reject AI suggestions (never auto-committing AI output silently) - Loading states specific to AI latency (which is often higher than a typical API call) - Handling failures/timeouts gracefully (LLM calls can fail or be slow more often than typical APIs) 4. **Prompt design for a specific feature** - Writing a focused system prompt for "break this task into subtasks" that reliably returns usable structured data - Testing prompts against edge cases (very vague input, very long input)

Recommended learning order: Backend AI-service layer pattern → streaming to frontend → UX design for AI output → prompt design/testing for the specific feature → wiring it all into the capstone.

Mandatory: All of the above. **Optional:** Multi-turn conversational AI features (this week focuses on single-purpose AI features; full chat interfaces are a natural next step post-roadmap).

Common beginner mistakes: - Auto-saving AI-generated content directly without a user review/confirmation step. - Not handling the "AI call failed or timed out" case at all, leaving the UI stuck. - Writing a vague prompt and being surprised the structured output is inconsistent — prompts need iteration and testing just like code.

Free resources: Anthropic's official docs on streaming and structured outputs, MDN's Server-Sent Events guide, real product UX patterns from Linear/Notion's public engineering blog posts on AI feature design (free).

Estimated time: 12 hours.

Week 42 — RAG Fundamentals

Build: A standalone "Docs Q&A" tool: ingest a handful of markdown files, generate embeddings, store them in a simple vector store, and answer questions grounded in that content (a minimal but complete RAG pipeline).

Capstone update: Add "AI-powered project search" to TaskForge AI: index a project's tasks/comments as embeddings, and let users ask natural-language questions like "what's blocking the launch?" answered using retrieval over their actual project data (not the model's general knowledge).

Learn: Retrieval-Augmented Generation — how to ground LLM responses in your own data, which is the core pattern behind most real-world "AI search" and "chat with your data" features.

Why: RAG is explicitly on your goal list and is one of the most commonly requested AI-engineering skills in 2026 product roles.

Detailed topics: 1. **Embeddings (practitioner level)** - What an embedding is (a vector representation of meaning) — conceptual, not the underlying math - Generating embeddings via an API - Similarity search (cosine similarity, conceptual understanding) 2. **Vector storage** - What a vector database does differently from a normal database - Options at a practical level: a lightweight local option (for learning) vs. a managed vector DB (e.g., Pinecone, or Postgres with `pgvector` — a genuinely practical choice since you already run Postgres) 3. **The RAG pipeline** - Chunking strategy (why you split documents, common chunking approaches) - Retrieval step: embed the query, search for relevant chunks - Augmentation: injecting retrieved chunks into the LLM's context as grounding - Generation: the LLM answers using the retrieved context, not just its training data 4. **RAG quality & limitations** - Why retrieval quality matters more than model quality for RAG accuracy - Common failure modes: irrelevant chunks retrieved, chunks too small/large, stale embeddings after data changes

Recommended learning order: Embeddings concept → similarity search → vector storage options → chunking strategy → full pipeline (retrieve → augment → generate) → quality/limitations awareness.

Mandatory: All of the above, with `pgvector` recommended for the capstone since it avoids adding a whole new infrastructure piece. **Optional:** Advanced retrieval techniques (hybrid search, re-ranking) — valuable to explore once the basic pipeline works well.

Common beginner mistakes: - Treating RAG as "just embed everything and it works" without thinking about chunking strategy, which is where most real-world RAG quality problems come from. - Not re-indexing embeddings when the underlying data changes, leading to stale/wrong answers. - Retrieving too much or too little context, either overwhelming the model or starving it of relevant information.

Free resources: Anthropic's official documentation on contextual retrieval/RAG patterns, `pgvector` official GitHub docs (free, thorough), Pinecone's free "Learning Center" articles on RAG fundamentals (vendor content, but genuinely educational and free).

Estimated time: 12 hours.

Week 43 — AI Workflows, Agents & Responsible AI Use

Build: Extend the Week 40 LLM script into a very small "agent": given a goal (e.g., "research and summarize"), let the LLM decide to call one of 2–3 defined tools (e.g., a web search function, a calculator function) via function calling, and observe/log its reasoning steps.

Capstone update: Add a lightweight "AI assistant" workflow to TaskForge AI that combines what you've built: given a natural-language request ("summarize what's overdue and suggest priorities"), it retrieves relevant data (RAG from Week 42), reasons over it, and can optionally call a tool (e.g., "create a task" via function calling) — with a clear user-facing confirmation step before any tool call actually mutates data.

Learn: The building blocks of AI agents/workflows (tool use, multi-step reasoning) at a practical level, and how to use AI responsibly in a real product.

Why: AI workflows/agents are the natural evolution of the AI features you've built so far, and understanding them (even at a basic level) is increasingly expected of software engineers working with AI.

Detailed topics: 1. **AI workflows vs. agents (conceptual distinction)** - A workflow: a fixed sequence of AI + deterministic steps - An agent: the LLM makes decisions about what

to do next (including which tools to call) - Why “agent” is overused in marketing but the underlying pattern (tool use + reasoning loop) is genuinely useful 2. **Function/tool calling in practice** - Defining tools with clear schemas the model can reliably call - Handling the model’s tool-call request, executing it, returning the result - Multi-step loops (call tool → observe result → decide next step) 3. **Guardrails for agentic features** - Never letting an AI agent perform destructive/irreversible actions without human confirmation (directly relevant to your capstone’s “create a task” tool call) - Setting reasonable limits (max steps, timeouts) to avoid runaway loops 4. **Using AI responsibly as a practicing engineer** - Being transparent about AI-generated content in your product (where relevant) - Data privacy considerations when sending user data to an LLM API - Bias/reliability awareness — LLMs can be confidently wrong, and product design should account for that

Recommended learning order: Workflow vs. agent distinction → tool-calling mechanics hands-on → building a minimal reasoning loop → guardrails/confirmation patterns → responsible-use principles applied to your own feature.

Mandatory: Tool-calling mechanics, guardrails/human-confirmation pattern, responsible-use awareness. **Optional:** Building a fully autonomous multi-agent system (a deep, fast-moving specialization — well beyond “software engineer who builds with AI,” and explicitly out of scope per your own goals).

Common beginner mistakes: - Letting an AI agent take irreversible actions (like deleting data) without any human-in-the-loop confirmation. - Not setting any step/time limits, risking runaway loops that burn API cost for no benefit. - Overselling “AI agent” in a portfolio when the actual implementation is a simple two-step workflow — be precise and honest about what you built, since interviewers will ask follow-up questions.

Free resources: Anthropic’s official documentation on tool use and building effective agents (Anthropic has published a well-regarded free engineering blog post specifically titled “Building Effective Agents” — genuinely worth reading in full), OWASP’s emerging guidance on LLM application security (free).

Estimated time: 10 hours.

Phase 7 milestone check: TaskForge AI now has genuine, working AI features — grounded search (RAG), structured AI generation, and a basic tool-using workflow — built responsibly with human-in-the-loop guardrails. This is a real, differentiated, 2026-relevant capstone.

Phase 8 — Capstone Finalization, Portfolio, Interview Prep (Weeks 44–46)

Week 44 — Capstone Finalization

Build/Update: No new mini project this week — full focus on finishing TaskForge AI to a genuinely production-quality standard: fix known bugs, complete any unfinished features from earlier weeks, do a final accessibility pass, finalize the README with screenshots/GIFs and a live demo link, and record a 2–3 minute demo video walkthrough.

Learn: How to take a project from "functionally complete" to "genuinely ready to show a stranger" — an underrated skill.

Why: A half-finished project with rough edges undersells months of real work. This week is entirely about polish and presentation, which materially affects how your work is perceived.

Detailed topics: 1. **Bug triage & fixing** - Going through the app systematically (every page, every action) as a skeptical user would 2. **Final accessibility & responsiveness pass** - Keyboard navigation check, mobile responsiveness check across the whole app (not just the landing page) 3. **Documentation finalization** - A README that clearly states: what it is, why you built it, tech stack, architecture diagram, how to run it locally, live demo link, and what you'd do next with more time 4. **Demo recording** - A short, well-narrated walkthrough video — genuinely useful for applications where a live link alone isn't enough context

Recommended learning order: Systematic bug pass → accessibility/responsiveness pass → README finalization → demo video recording.

Mandatory: All of the above. **Optional:** N/A — this is the finishing week, everything here matters.

Common beginner mistakes: - Treating "it works for the happy path I always test" as "finished," and never trying to break your own app. - Submitting a portfolio project with a broken/outdated README that doesn't match the current state of the code. - Skipping the demo video/screenshots because the live link "should be enough" — many reviewers won't take the time to run it themselves.

Free resources: WAVE (free browser accessibility checker), Loom's free tier for demo recording, "Make a README" (revisited from Week 34).

Estimated time: 12 hours.

Week 45 — Open Source Basics & Technical Writing Portfolio

Build: Make one genuine open-source contribution (a documentation fix, a small bug fix, or a well-scoped "good first issue") to a real public repository. Write one technical blog post (published on a free platform like Dev.to or a personal site) explaining a genuinely hard problem you solved in TaskForge AI (e.g., "How I designed RAG-powered search for my project management app").

Capstone update: N/A this week — this week is about visibility and communication artifacts around the capstone, not new code.

Learn: How open-source contribution actually works in practice, and how to write technically about your own work — both are concrete, checkable signals to employers.

Why: A real (even small) open-source PR and a genuine technical blog post are disproportionately effective portfolio signals — they show initiative and communication skill, not just

coding ability.

Detailed topics: 1. **Open source contribution workflow** - Finding a well-scoped first issue (labels like “good first issue,” “help wanted”) - Forking, branching, making a focused change, writing a clear PR description - Responding to review feedback professionally 2. **Technical writing** - Structuring a technical post: problem → why it’s hard → your approach → what you learned - Writing for clarity over cleverness (the same “clean code” instinct, applied to prose) - Including real code snippets/diagrams where they clarify, not just to pad length

Recommended learning order: Find and understand an open-source contribution workflow → make one real contribution → outline a technical post about your hardest capstone problem → write and publish it.

Mandatory: One real open-source contribution attempt, one published technical post. **Optional:** Ongoing/ regular open-source contribution (excellent long-term habit, not required this week).

Common beginner mistakes: - Picking a contribution that’s too ambitious for a first PR and getting stuck/discouraged. - Writing a technical post that’s really just a feature list instead of explaining a genuine problem and your reasoning. - Never actually publishing/sharing the post out of perfectionism.

Free resources: “First Contributions” (free GitHub repo designed to teach the OSS contribution workflow), Dev.to (free publishing platform), GitHub’s own “How to Contribute to Open Source” guide.

Estimated time: 8–10 hours.

Week 46 — Interview Preparation & Job Application Strategy

Build: No new project — a mock system design interview (recorded, self-reviewed or done with a peer) using TaskForge AI’s own architecture as the subject, plus a mock behavioral interview.

Capstone update: Prepare a concise, confident 2-minute “walk me through a project” narrative for TaskForge AI, and prepare answers to the architectural decisions you documented in your ADRs (Week 29) — you should be able to defend every major decision.

Learn: How to translate 46 weeks of real work into strong interview performance — technical, behavioral, and portfolio presentation.

Why: All the skill in the world doesn’t matter if it can’t be communicated clearly under interview conditions. This final week is where preparation becomes results.

Detailed topics: 1. **Technical interview prep** - Reviewing core JS/TS/React/Node concepts as *explain it out loud* practice, not just recognition - Practicing a system design walkthrough using your own capstone as the example (you already have the ADRs from Week 29) 2. **Behavioral interview prep** - STAR method (Situation, Task, Action, Result) for structuring answers - Preparing genuine stories from your roadmap journey (a real bug you struggled with, a real architecture decision you changed your mind on) - Addressing the 3-year gap honestly and confidently — framing it as focused, structured skill-building (which it genuinely was) backed by a real, live, tested, deployed project as proof 3. **Portfolio presentation** - A clean, simple personal site/portfolio page linking to TaskForge AI (live demo, GitHub, blog post, and 1–2 other mini projects) - Tailoring your resume to highlight concrete, specific technologies and outcomes (not vague buzzwords) 4. **Job application strategy** - Where to actually apply in 2026 (startup job boards, company career pages, referrals via your new open-source/blog

visibility) - Realistic expectations: application volume, follow-up cadence, using rejections as calibration, not just discouragement

Recommended learning order: Technical concept review (spoken, not just read) → system design mock walkthrough → behavioral story prep (STAR) → portfolio page assembly → resume tailoring → application strategy planning.

Mandatory: All of the above. **Optional:** N/A — every part of this week directly serves your stated goal of becoming employable.

Common beginner mistakes: - Preparing answers about concepts in the abstract instead of anchoring them in your own, real project — concrete beats generic every time. - Avoiding the employment-gap question instead of addressing it directly and confidently with the evidence you now have. - Applying only to a handful of "dream" companies instead of a realistic, broad, consistent pipeline.

Free resources: "The System Design Primer" (revisited), free mock-interview platforms (e.g., Pramp), "STAR Method" guides (widely available for free from reputable career-advice sources).

Estimated time: 12 hours.

Phase 8 milestone check — Roadmap complete. You now have: a live, tested, secure, AI-powered, full-stack TypeScript application; a documented architecture with real decision records; a public technical writing sample; a real open-source contribution; and rehearsed interview material — all built, not copied.

Capstone Project Evolution — “TaskForge AI”

Weeks	State of the Capstone
1–4	Repo created, spec written, no UI yet
5–6	TypeScript + tooling set up (ESLint/Prettier/strict tsconfig)
7–9	Static, responsive, Tailwind-styled landing page
10–16	Full interactive React + TS frontend against mock data, tested with Vitest/RTL
17–20	Real Node/Express/TypeScript backend, PostgreSQL + Prisma, first real persisted data
21	Polyglot persistence: MongoDB added for flexible activity-log data
22	Real authentication (JWT) and authorization (RBAC)
23–25	Validated, documented, layered, tested backend architecture
26	File uploads, third-party email API, background job pattern
27–31	Clean code pass, applied design patterns, written ADRs, system design doc
32–34	Redis caching, security hardening pass, performance audit, strong README
35–38	Fully deployed to production with CI/CD and Docker-based local dev
39–43	Real AI features: AI task breakdown, RAG-powered project search, tool-using AI workflow with human-in-the-loop guardrails
44–46	Fully polished, documented, demoed, and interview-ready

Job Readiness, Portfolio, and Interview Milestones

Portfolio milestones: - Week 16: A polished, responsive, tested frontend — shareable even before the backend exists. - Week 26: A complete full-stack application with real auth, persistence, and third-party integrations. - Week 36: A live, deployed application with a real URL. - Week 43: A genuinely differentiated AI-powered feature set (RAG + agentic workflow) most bootcamp portfolios won't have. - Week 46: A finished portfolio site, a technical blog post, and one open-source contribution.

Job readiness milestones: - Week 20: Can build and reason about a real relational schema and CRUD API. - Week 26: Can build a complete, secure, tested full-stack feature independently. - Week 31: Can hold a basic system design conversation using real vocabulary (scalability, caching, load balancing). - Week 38: Can deploy, containerize, and set up CI/CD for a real application — genuine "can be productive on day one" signal. - Week 43: Can speak credibly about building with AI/LLMs — a 2026-specific differentiator.

Interview milestones: - Week 29 & 31: First real practice explaining architecture decisions out loud (ADRs, system design framework). - Week 33: Can discuss security tradeoffs concretely, using your own project as evidence. - Week 46: Full mock technical + behavioral interview practice, portfolio narrative rehearsed, application pipeline started.

Final Revision Plan (Post-Week 46)

Treat this as an ongoing 2-week rotation once the roadmap is complete and you're actively applying:

Week A (odd weeks): - 2–3 hours: Revisit one weak area flagged during mock interviews (e.g., a system design topic, a specific hook, SQL joins). - 2–3 hours: Contribute to open source again, or fix a real bug/add a small feature to TaskForge AI (keeps the project "alive" and gives you fresh talking points). - Continue applying to roles consistently; log every application and outcome.

Week B (even weeks): - 2–3 hours: Do one full mock interview (technical or system design), self-recorded or with a peer. - 2–3 hours: Read/watch one piece of current engineering content (an engineering blog post from a company you admire, a recent conference talk) to stay current post-roadmap. - Review and refine your resume/portfolio based on the last two weeks of interview feedback.

Ongoing habit: Keep doing your existing daily DSA/LeetCode practice — this roadmap was intentionally designed around it, not instead of it.

Roadmap Ratings

Category	Score (out of 10)	Rationale
Beginner Friendliness	8/10	Starts from true fundamentals with no assumed React/Node knowledge; heavy scaffolding early. Not a 9–10 only because the pace (12–15 hrs/week for 46 weeks) still demands real discipline from a near-beginner.
Job Readiness	9/10	Covers the full 2026 hiring bar for junior/mid full-stack roles: TS, testing, auth, deployment, CI/CD, security, and system design fundamentals — not just "build a CRUD app."
Interview Readiness	8/10	Dedicated system design, architecture (ADR), and interview-prep weeks, plus constant "explain why" framing throughout. Deliberately doesn't replace your separate DSA practice, which remains the other half of technical interview readiness.
Portfolio Value	9/10	One deep, evolving, deployed, tested, AI-powered capstone beats five shallow tutorial clones — and the AI features (RAG + agentic workflow) are a genuine 2026 differentiator most junior portfolios lack.
AI-Era Relevance	9/10	Five full weeks on practical AI engineering (assisted development, LLM APIs, AI features, RAG, agents/responsible use) without drifting into AI-research territory you explicitly don't want.

Category	Score (out of 10)	Rationale
Completion Probability	7/10	This is the honest number: 46 weeks is a long, serious commitment. The structure (small weekly wins, one evolving project instead of scattered ones, progressive difficulty, generous buffer language at phase checkpoints) is specifically designed to maximize follow-through, but a roadmap this comprehensive always carries real attrition risk — protect it by treating consistency over 46 weeks as more important than intensity in any single week.

Overall note: This roadmap was deliberately optimized for *completion*, not maximum difficulty — per your explicit instruction. If you find any single week is taking meaningfully longer than its estimate, that's expected and fine; extend rather than skip. Skipping fundamentals to "stay on schedule" is the single most common way self-taught developers end up with an impressive-looking but shallow skill set.